

CSE 385: Analysis of Algorithms (Honors)

Lecture 8 (Divide-and-Conquer Algorithms: The Master Theorem)

**Rezaul A. Chowdhury
Department of Computer Science
SUNY Stony Brook
Spring 2026**

A Recurrence for Recursive Divide & Conquer

RECURSIVE-DIVIDE-AND-CONQUER (P, n)

INPUT: A problem P of size $n \geq 1$

OUTPUT: Solution S of P computed using recursive divide and conquer

1. **if** $n = 1$ **then**

2. $S \leftarrow$ solution of P computed directly w/o divide & conquer

3. **else**

4. DIVIDE:

5. divide P into a subproblems $P_1, P_2, \dots, P_a$ of size $\frac{n}{b}$ each

6. CONQUER:

7. **for** $i = 1$ **to** a **do**

8. $S_i \leftarrow$ RECURSIVE-DIVIDE-AND-CONQUER ($P_i, \frac{n}{b}$)

9. COMBINE:

10. $S \leftarrow$ solution of P obtained by combining $S_1, S_2, \dots, S_a$

11. **endif**

12. **return** S

$T(n)$

$\Theta(1)$

$f_d(n)$

$a \times T\left(\frac{n}{b}\right)$

$f(n)$

$f_c(n)$

$$T(n) = \begin{cases} \Theta(1), & \text{if } n \leq 1, \\ aT\left(\frac{n}{b}\right) + f(n), & \text{otherwise.} \end{cases}$$

A Useful Recurrence

Consider the following recurrence:

$$T(n) = \begin{cases} \Theta(1), & \text{if } n \leq 1, \\ aT\left(\frac{n}{b}\right) + f(n), & \text{otherwise;} \end{cases}$$

where, $a \geq 1$ and $b > 1$ are constants (i.e., independent of n).

Arises frequently in the analyses of *divide-and-conquer* algorithms.

Consider the following recurrences.

Karatsuba's Integer Multiplication Algorithm: $T(n) = 3T\left(\frac{n}{2}\right) + \Theta(n)$

Strassen's Matrix Multiplication Algorithm: $T(n) = 7T\left(\frac{n}{2}\right) + \Theta(n^2)$

Fast Fourier Transform: $T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)$

How the Recurrence Unfolds

$$T(n) = \begin{cases} \Theta(1), & \text{if } n \leq 1, \\ aT\left(\frac{n}{b}\right) + f(n), & \text{otherwise.} \end{cases}$$

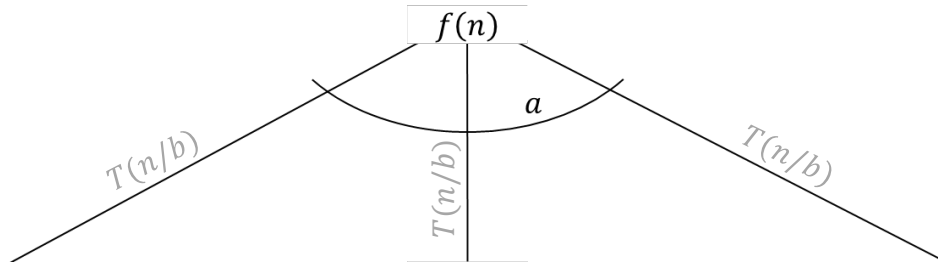
How the Recurrence Unfolds

$$T(n) = \begin{cases} \Theta(1), & \text{if } n \leq 1, \\ aT\left(\frac{n}{b}\right) + f(n), & \text{otherwise.} \end{cases}$$

$T(n)$

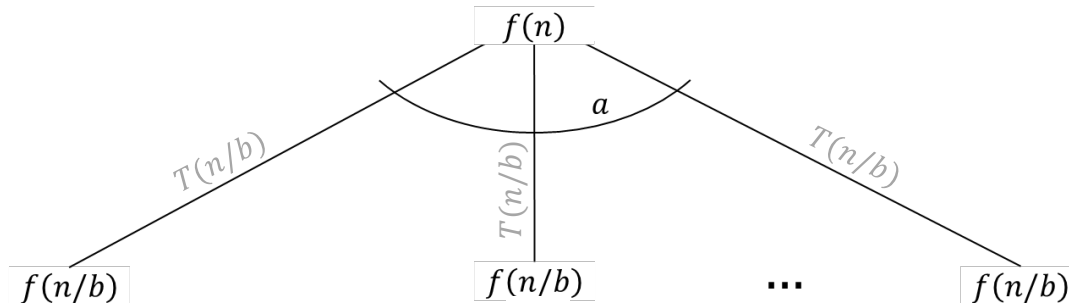
How the Recurrence Unfolds

$$T(n) = \begin{cases} \Theta(1), & \text{if } n \leq 1, \\ aT\left(\frac{n}{b}\right) + f(n), & \text{otherwise.} \end{cases}$$



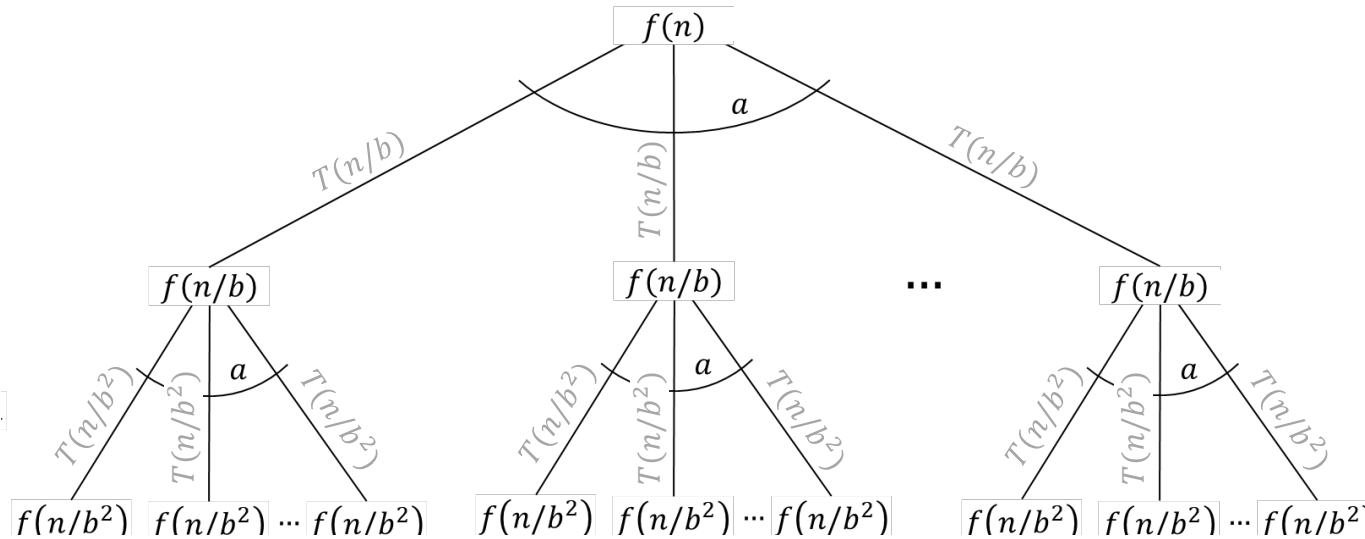
How the Recurrence Unfolds

$$T(n) = \begin{cases} \Theta(1), & \text{if } n \leq 1, \\ aT\left(\frac{n}{b}\right) + f(n), & \text{otherwise.} \end{cases}$$



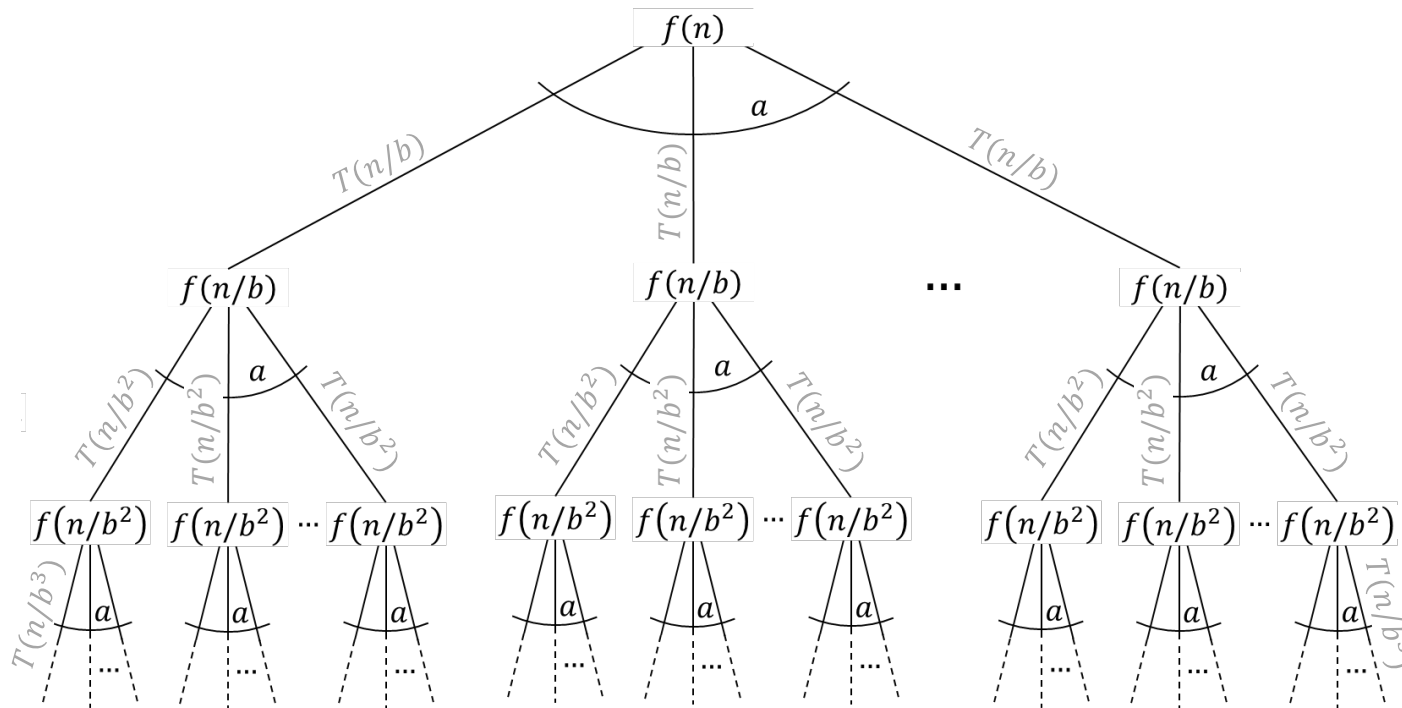
How the Recurrence Unfolds

$$T(n) = \begin{cases} \Theta(1), & \text{if } n \leq 1, \\ aT\left(\frac{n}{b}\right) + f(n), & \text{otherwise.} \end{cases}$$



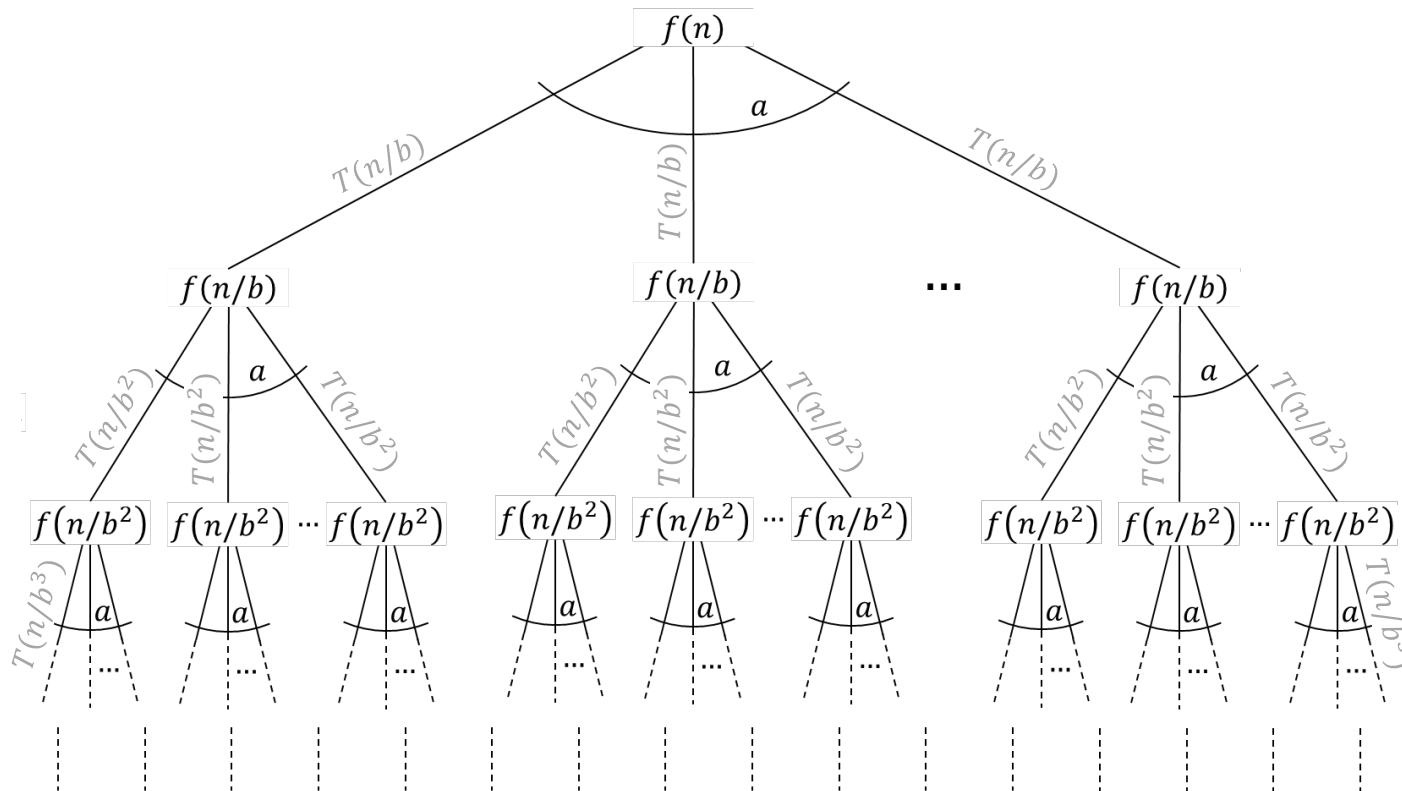
How the Recurrence Unfolds

$$T(n) = \begin{cases} \Theta(1), & \text{if } n \leq 1, \\ aT\left(\frac{n}{b}\right) + f(n), & \text{otherwise.} \end{cases}$$



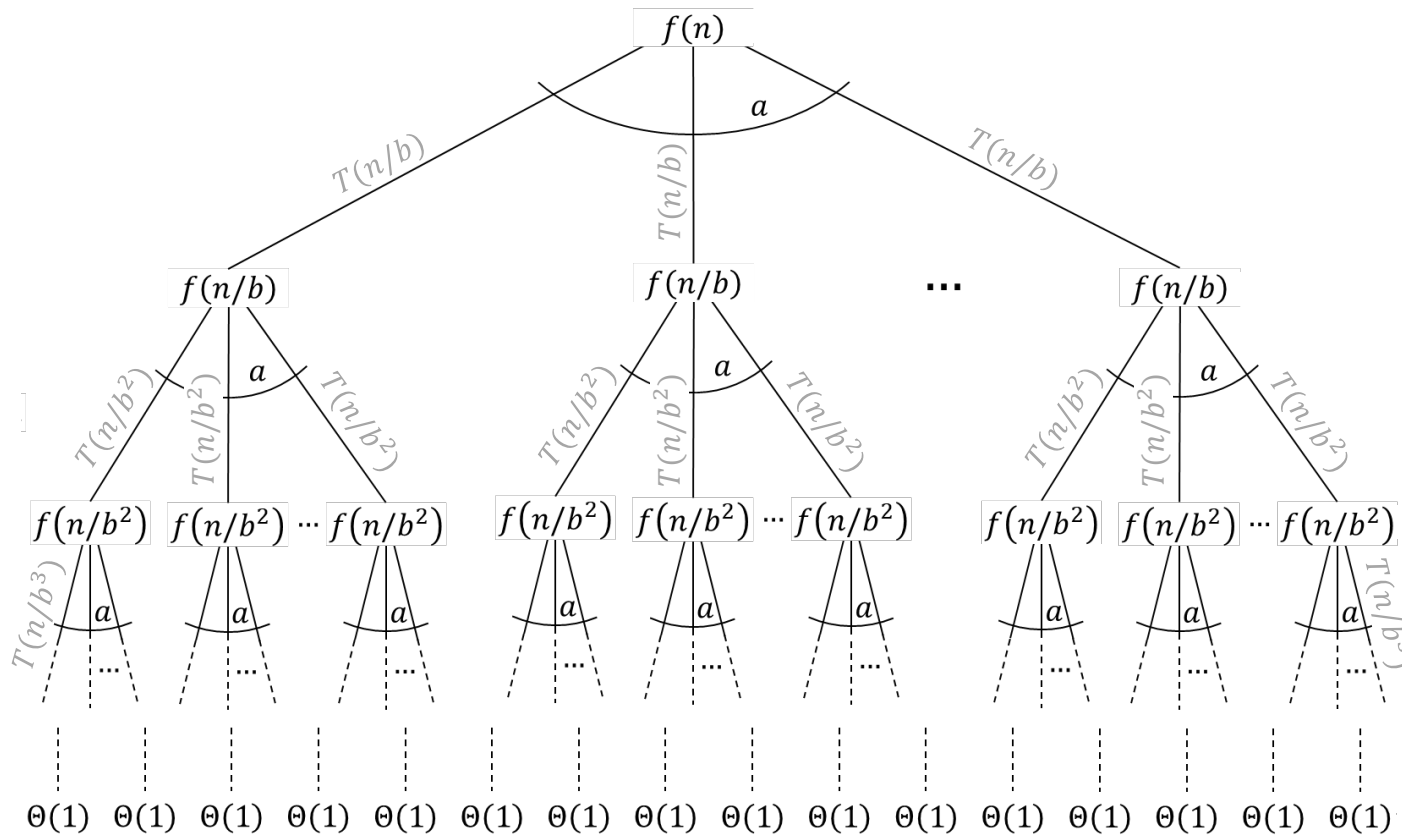
How the Recurrence Unfolds

$$T(n) = \begin{cases} \Theta(1), & \text{if } n \leq 1, \\ aT\left(\frac{n}{b}\right) + f(n), & \text{otherwise.} \end{cases}$$



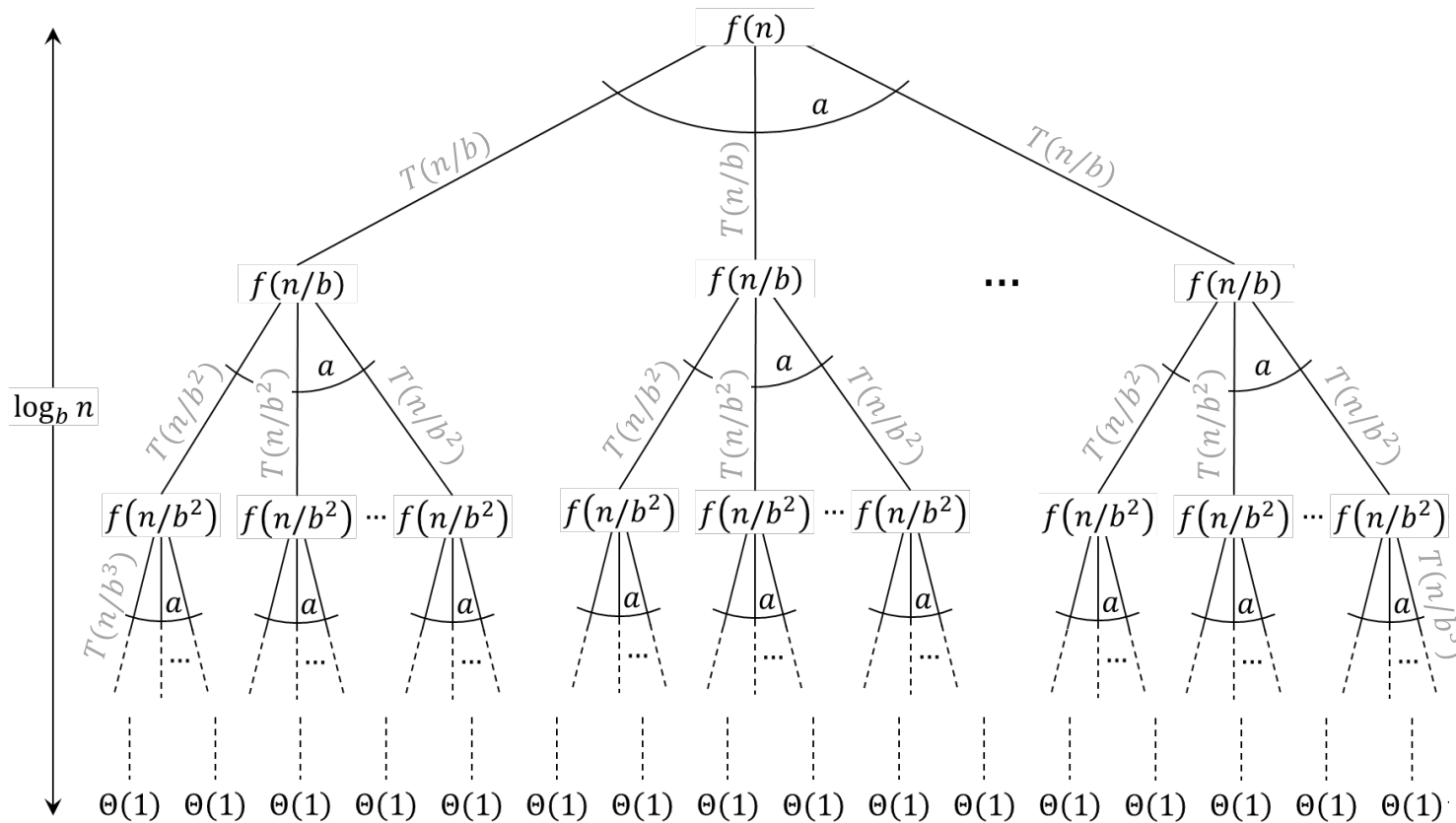
How the Recurrence Unfolds

$$T(n) = \begin{cases} \Theta(1), & \text{if } n \leq 1, \\ aT\left(\frac{n}{b}\right) + f(n), & \text{otherwise.} \end{cases}$$



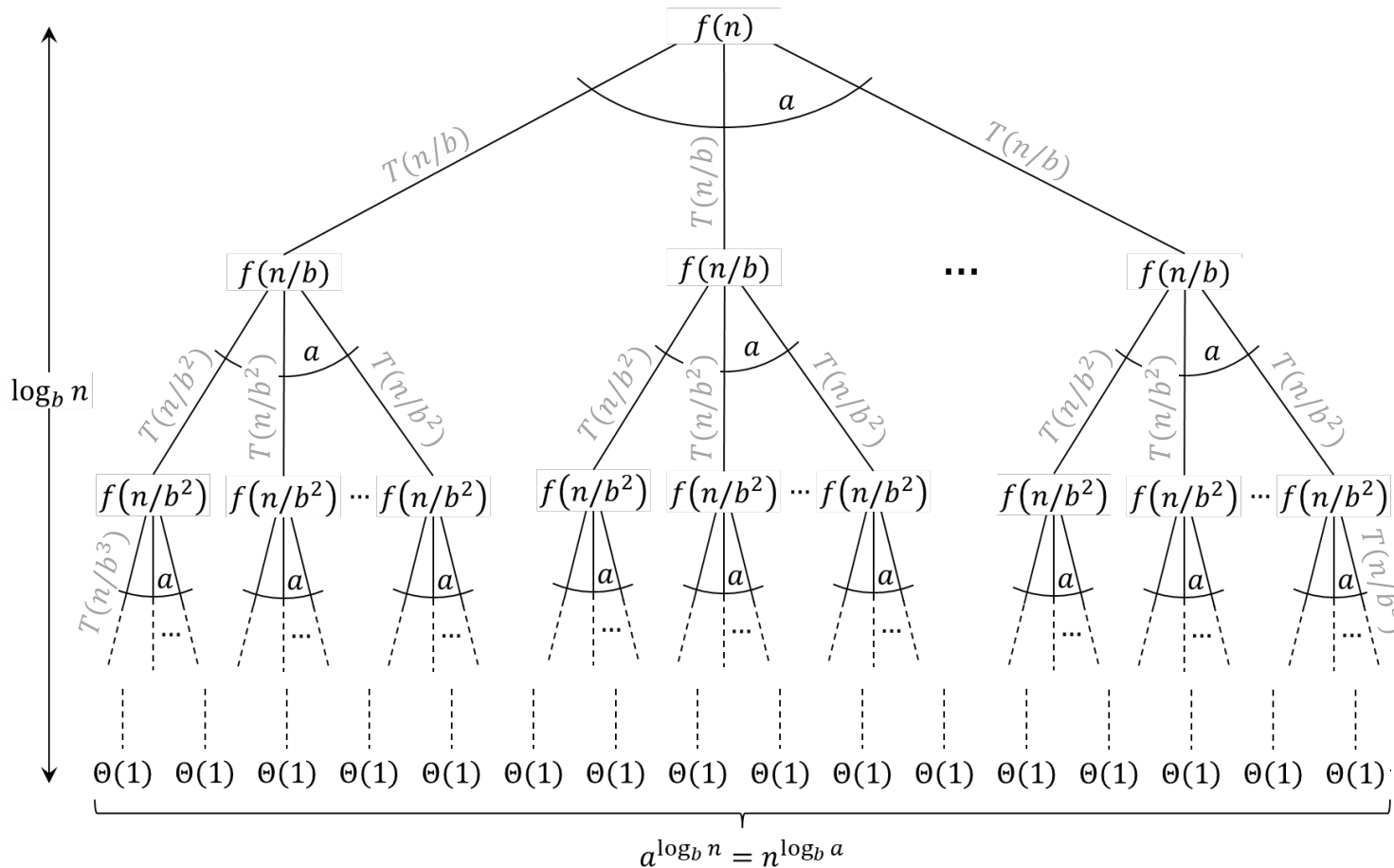
How the Recurrence Unfolds

$$T(n) = \begin{cases} \Theta(1), & \text{if } n \leq 1, \\ aT\left(\frac{n}{b}\right) + f(n), & \text{otherwise.} \end{cases}$$



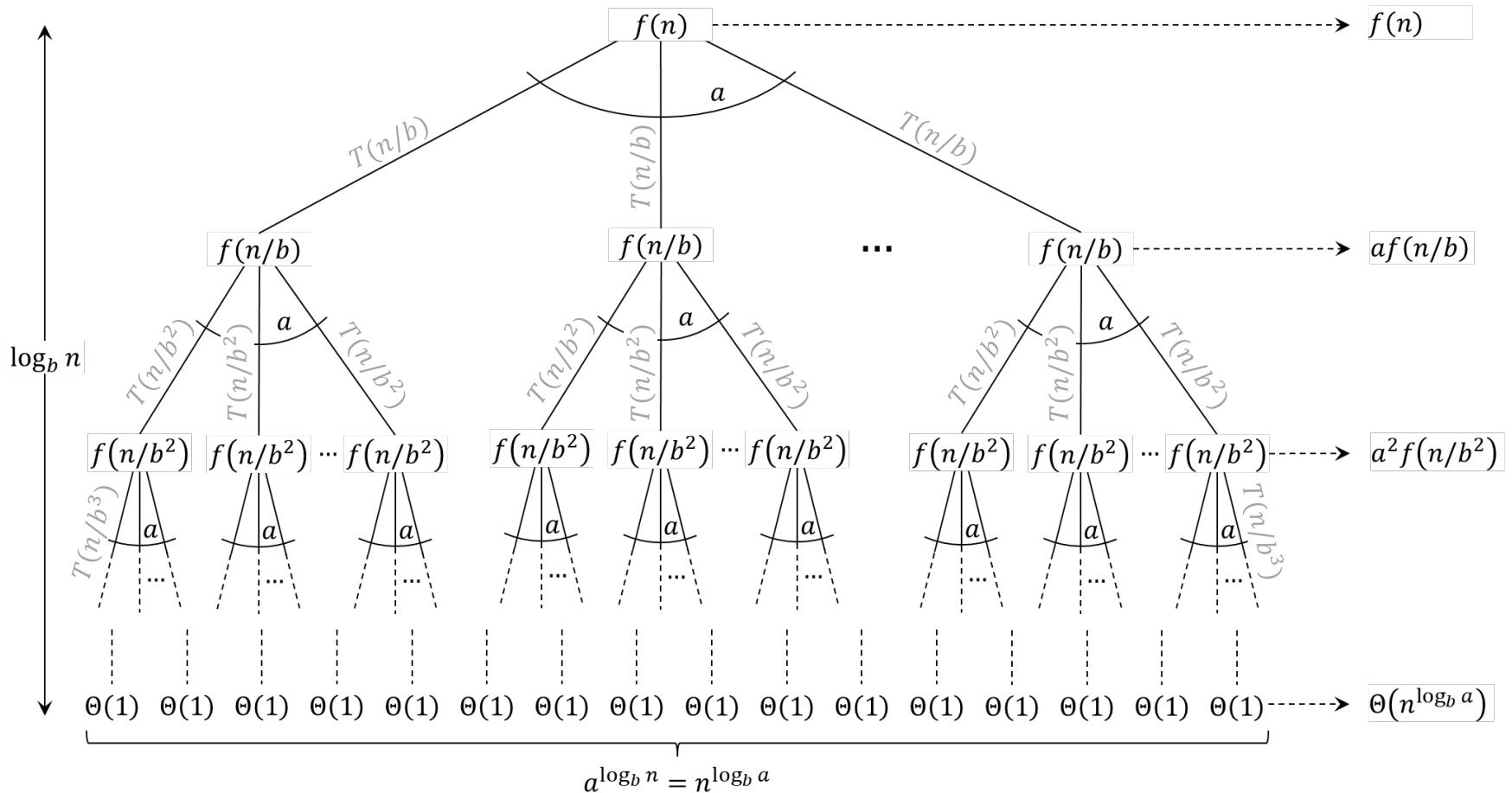
How the Recurrence Unfolds

$$T(n) = \begin{cases} \Theta(1), & \text{if } n \leq 1, \\ aT\left(\frac{n}{b}\right) + f(n), & \text{otherwise.} \end{cases}$$



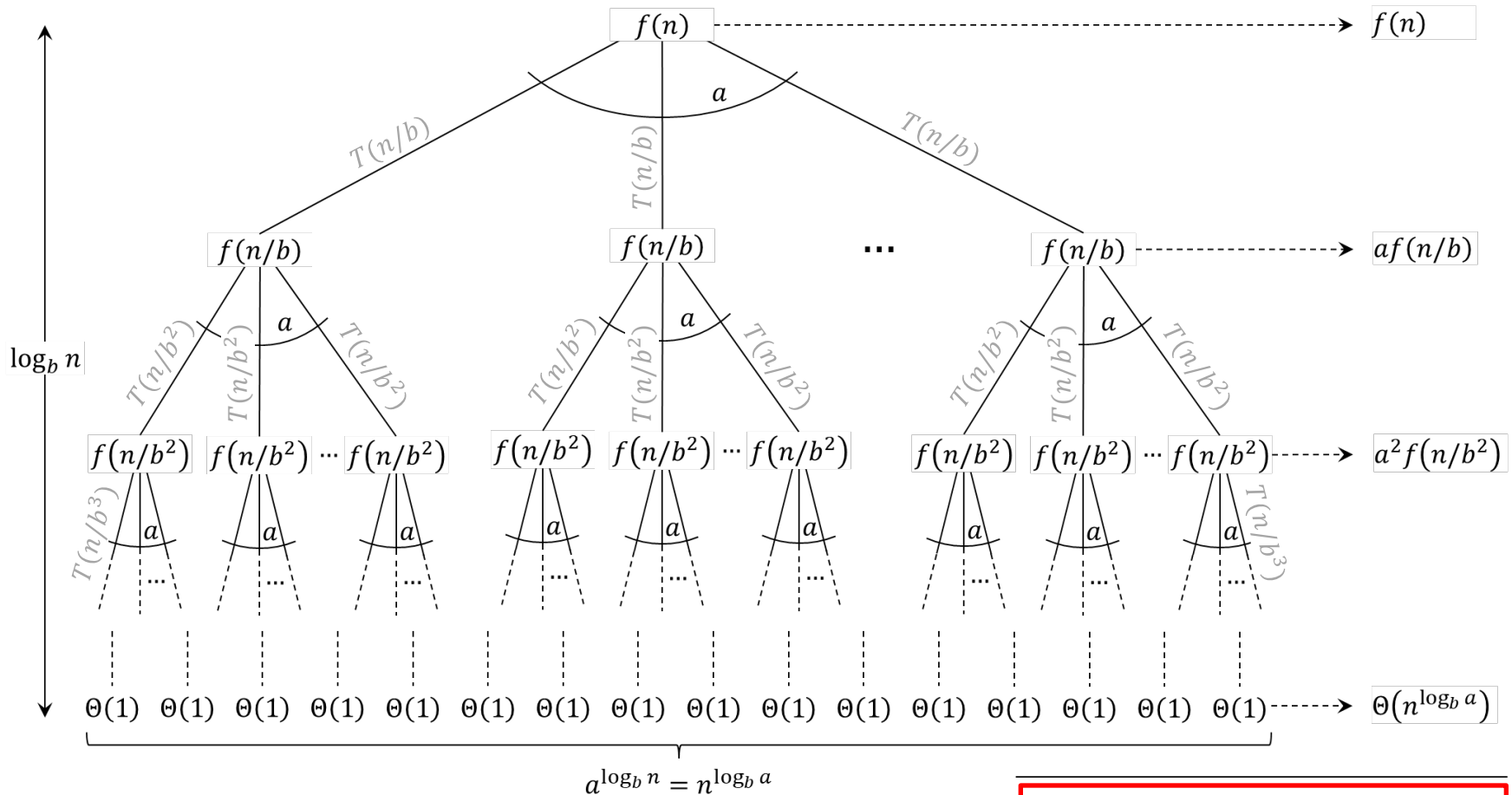
How the Recurrence Unfolds

$$T(n) = \begin{cases} \Theta(1), & \text{if } n \leq 1, \\ aT\left(\frac{n}{b}\right) + f(n), & \text{otherwise.} \end{cases}$$



How the Recurrence Unfolds

$$T(n) = \begin{cases} \Theta(1), & \text{if } n \leq 1, \\ aT\left(\frac{n}{b}\right) + f(n), & \text{otherwise.} \end{cases}$$



$$\text{Total: } \Theta(n^{\log_b a}) + \sum_{j=0}^{\log_b n - 1} a^j f(n/b^j)$$

How the Recurrence Unfolds

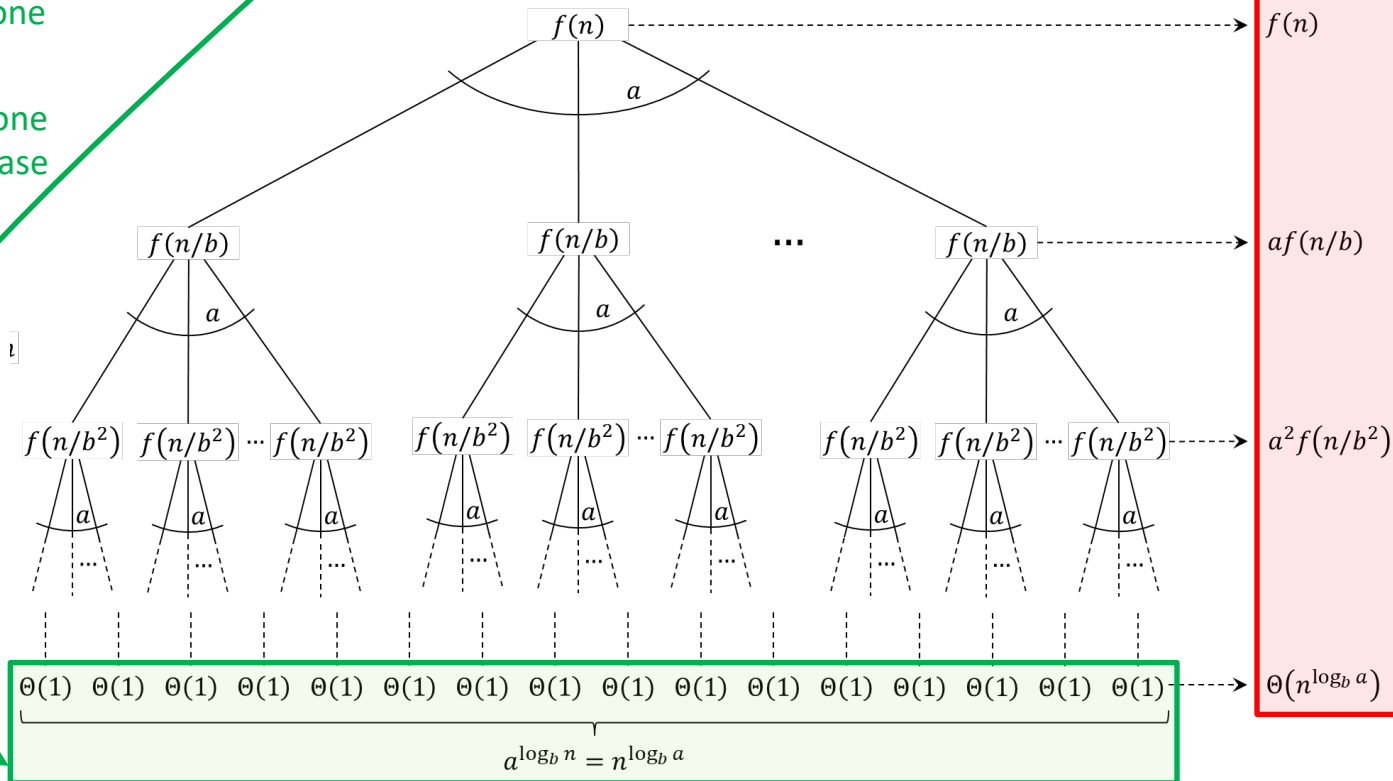
$$T(n) = \begin{cases} \Theta(1), & \text{if } n \leq 1, \\ aT\left(\frac{n}{b}\right) + f(n), & \text{otherwise.} \end{cases}$$

$$= \Theta(n^{\log_b a}) + \sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right)$$

total work done at internal nodes

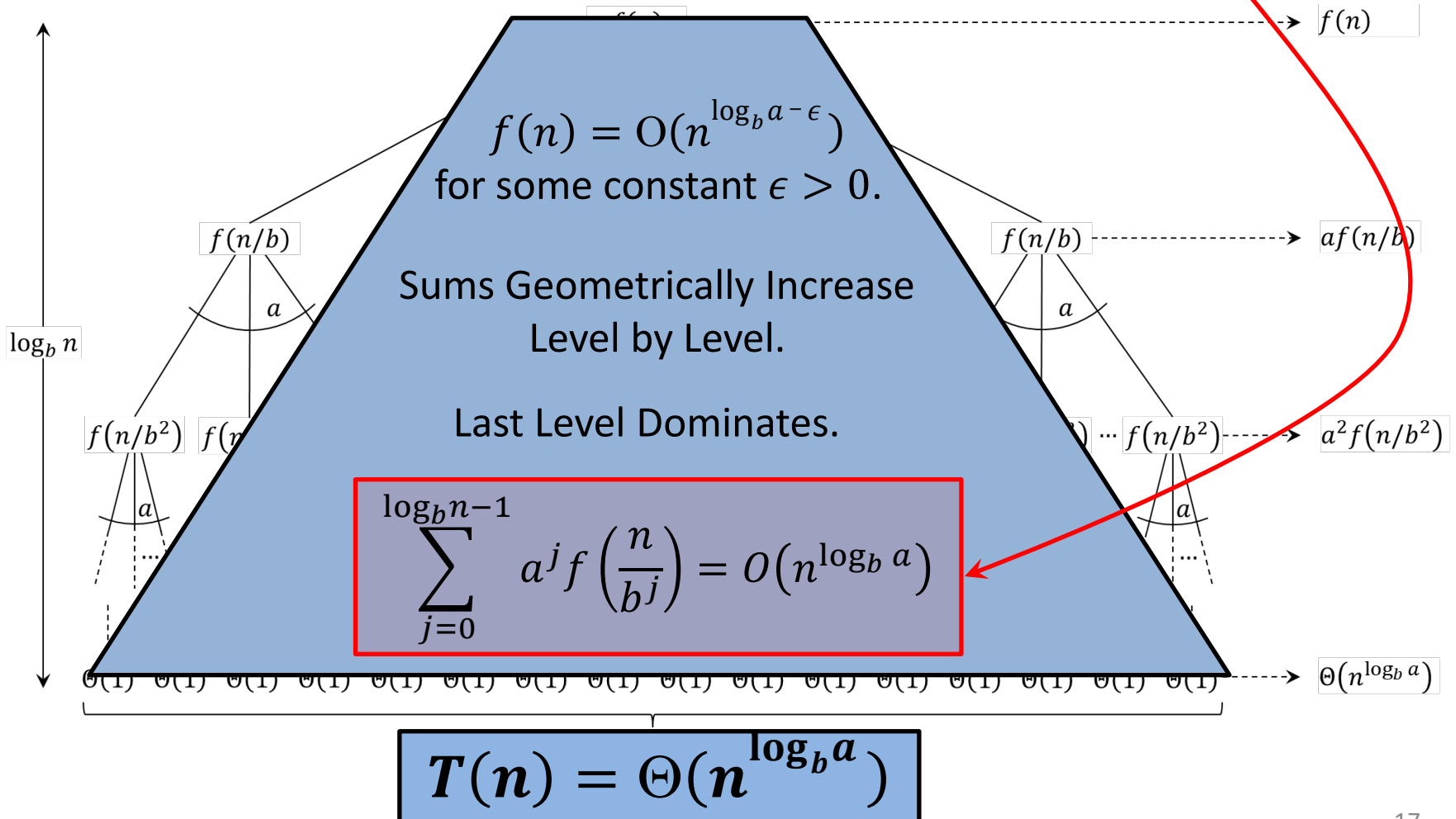
= total work done doing divide and combine

total work done at the leaves
= total work done at the base case



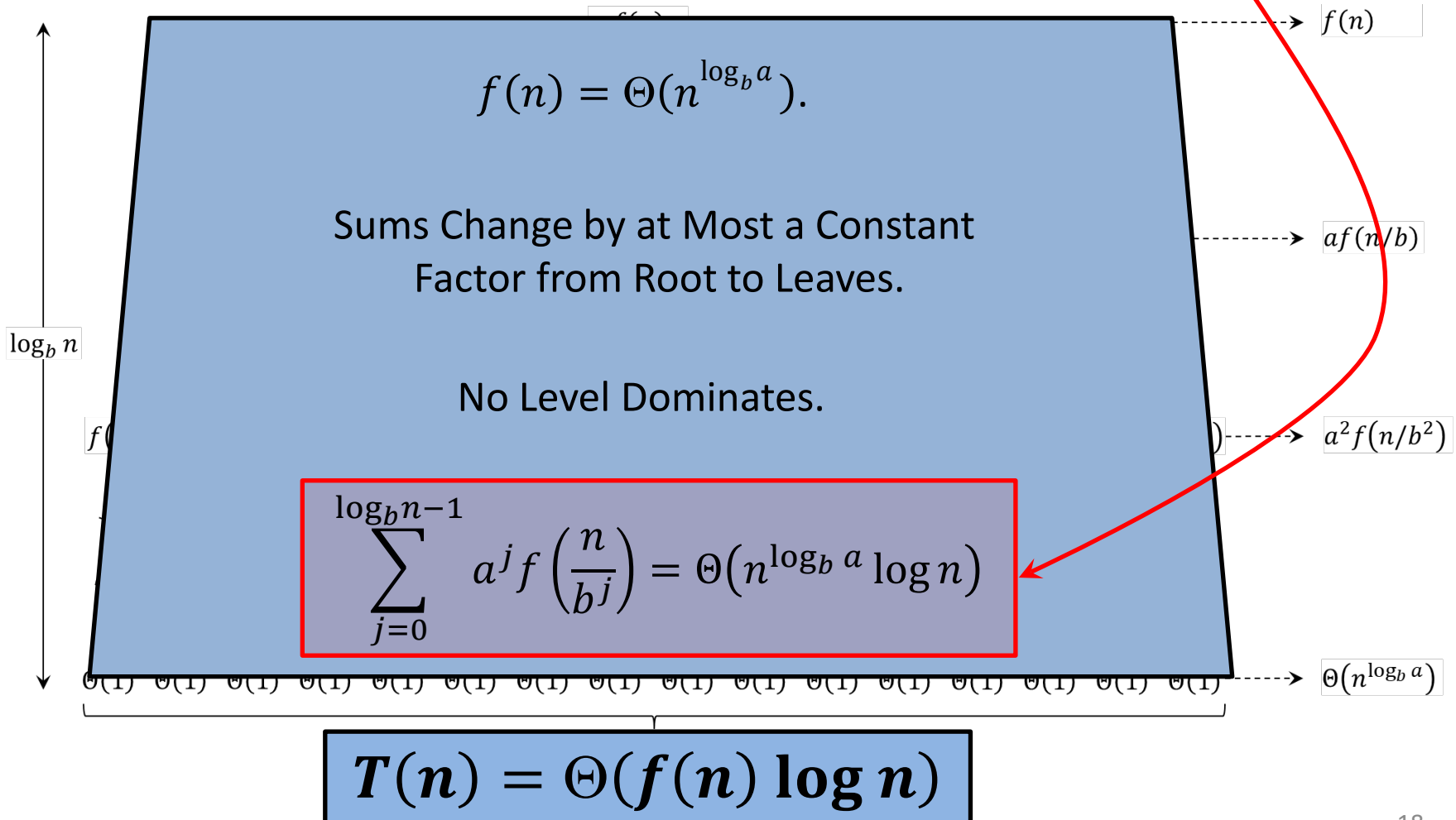
How the Recurrence Unfolds: Case 1

$$T(n) = \begin{cases} \Theta(1), & \text{if } n \leq 1, \\ aT\left(\frac{n}{b}\right) + f(n), & \text{otherwise.} \end{cases} = \boxed{\Theta(n^{\log_b a})} + \boxed{\sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right)}$$



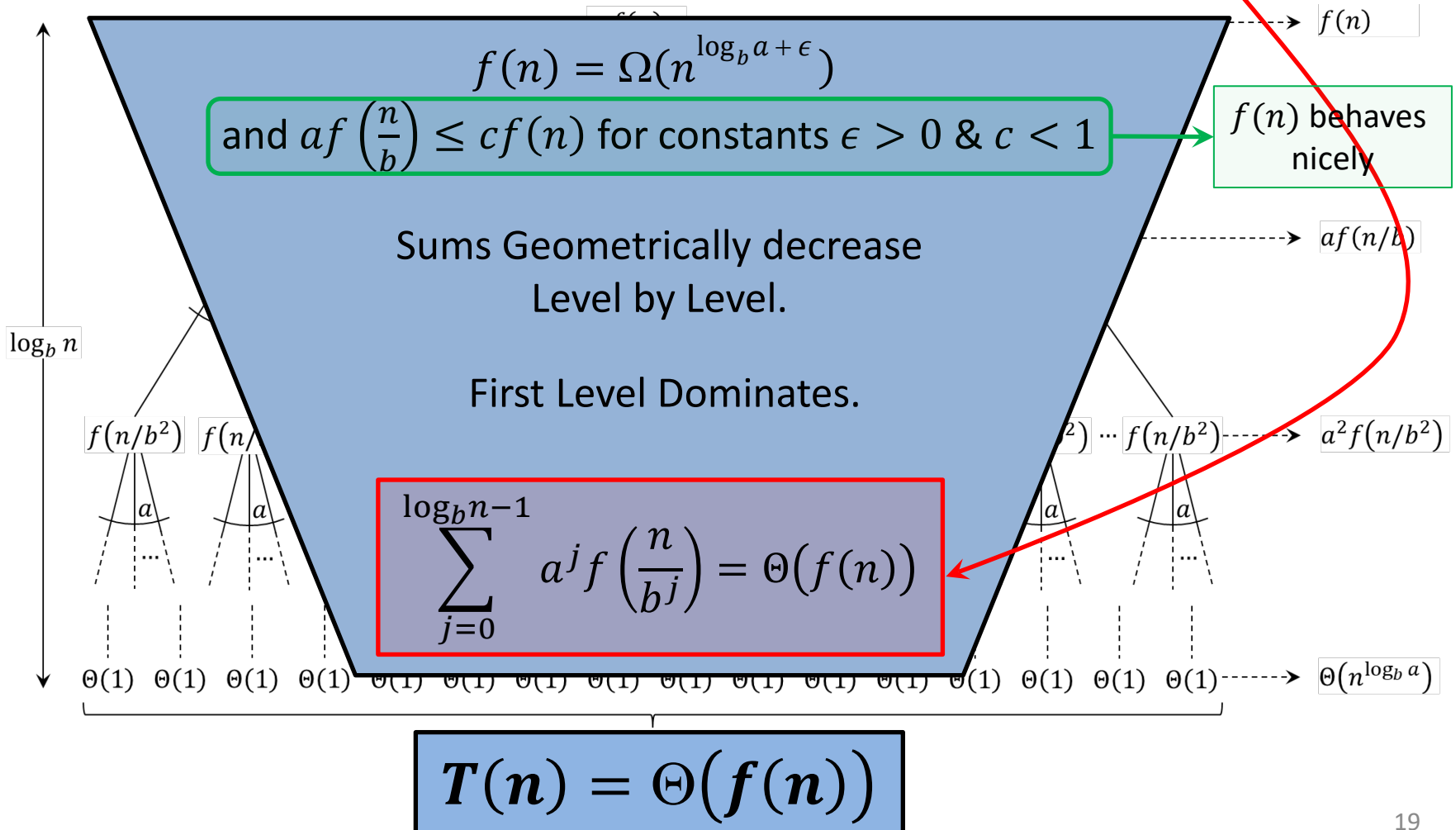
How the Recurrence Unfolds: Case 2

$$T(n) = \begin{cases} \Theta(1), & \text{if } n \leq 1, \\ aT\left(\frac{n}{b}\right) + f(n), & \text{otherwise.} \end{cases} = \boxed{\Theta(n^{\log_b a})} + \boxed{\sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right)}$$



How the Recurrence Unfolds: Case 3

$$T(n) = \begin{cases} \Theta(1), & \text{if } n \leq 1, \\ aT\left(\frac{n}{b}\right) + f(n), & \text{otherwise.} \end{cases} = \boxed{\Theta(n^{\log_b a})} + \sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right)$$



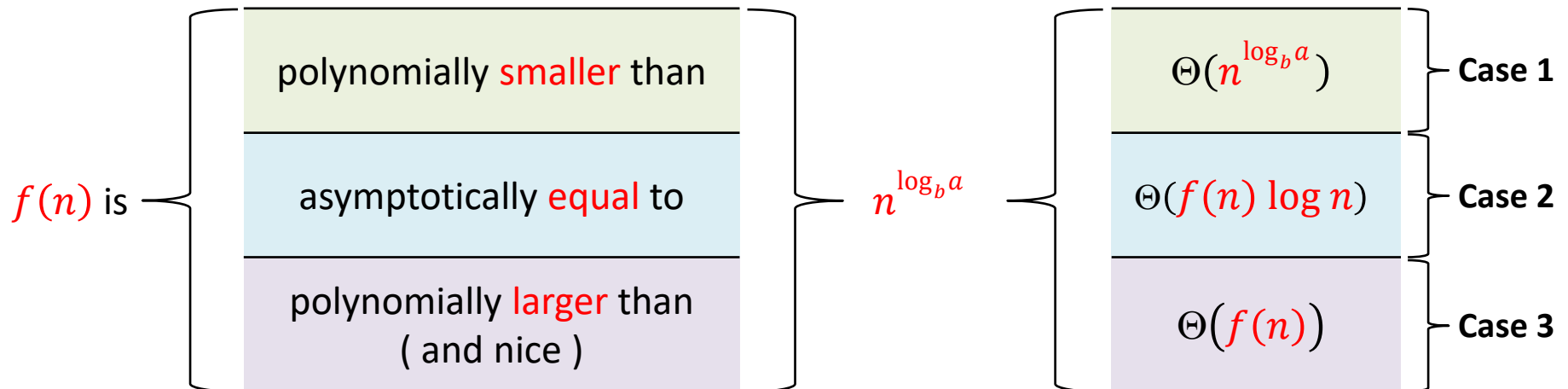
The Master Theorem

$$T(n) = \begin{cases} \Theta(1), & \text{if } n \leq 1, \\ aT\left(\frac{n}{b}\right) + f(n), & \text{otherwise } (a \geq 1, b > 1). \end{cases}$$

cost of divide and combine = $f(n)$
 number of leaves = $n^{\log_b a}$ } **compare**

if the relationship between $f(n)$ and $n^{\log_b a}$ is as follows

then $T(n)$ is



The Master Theorem

$$T(n) = \begin{cases} \Theta(1), & \text{if } n \leq 1, \\ aT\left(\frac{n}{b}\right) + f(n), & \text{otherwise } (a \geq 1, b > 1). \end{cases}$$

Case 1: $f(n) = O(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$

$$T(n) = \Theta(n^{\log_b a})$$

Case 2: $f(n) = \Theta(n^{\log_b a})$

$$T(n) = \Theta(f(n) \log n)$$

Case 3: $f(n) = \Omega(n^{\log_b a + \epsilon})$ and $af\left(\frac{n}{b}\right) \leq cf(n)$
for constants $\epsilon > 0$ and $c < 1$

$$T(n) = \Theta(f(n))$$

The Master Theorem

$$T(n) = \begin{cases} \Theta(1), & \text{if } n \leq 1, \\ aT\left(\frac{n}{b}\right) + f(n), & \text{otherwise } (a \geq 1, b > 1). \end{cases}$$

Case 1: $f(n) = O(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$

$$T(n) = \Theta(n^{\log_b a})$$

extended

Case 2: $f(n) = \Theta(n^{\log_b a} \log^k n)$ for some constant $k \geq 0$.

$$T(n) = \Theta(f(n) \log n)$$

Case 3: $f(n) = \Omega(n^{\log_b a + \epsilon})$ and $af\left(\frac{n}{b}\right) \leq cf(n)$
for constants $\epsilon > 0$ and $c < 1$

$$T(n) = \Theta(f(n))$$

Example Applications of Master Theorem

Example 1: $T(n) = 3T\left(\frac{n}{2}\right) + \Theta(n)$

Master Theorem Case 1: $T(n) = \Theta(n^{\log_2 3})$

Example 2: $T(n) = 7T\left(\frac{n}{2}\right) + \Theta(n^2)$

Master Theorem Case 1: $T(n) = \Theta(n^{\log_2 7})$

Example 3: $T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)$

Master Theorem Case 2: $T(n) = \Theta(n \log n)$

Assuming that we have an infinite number of processors, and all recursive calls in example 2 above can be executed in parallel:

Example 4: $T(n) = T\left(\frac{n}{2}\right) + \Theta(n^2)$

Master Theorem Case 3: $T(n) = \Theta(n^2)$

Recurrences not Solvable using the Master Theorem

Example 1: $T(n) = \sqrt{n} T\left(\frac{n}{2}\right) + n$

$a = \sqrt{n}$ is not a constant

Example 2: $T(n) = 2T\left(\frac{n}{\log n}\right) + n^2$

$b = \log n$ is not a constant

Example 3: $T(n) = \frac{1}{2} T\left(\frac{n}{2}\right) + n^2$

$a = \frac{1}{2}$ is not ≥ 1

Example 4: $T(n) = 2T\left(\frac{4n}{3}\right) + n$

$b = \frac{3}{4}$ is not > 1 .

Recurrences not Solvable using the Master Theorem

Example 5: $T(n) = 3T\left(\frac{n}{2}\right) - n$

$f(n) = -n$ is not positive

Example 6: $T(n) = 2T\left(\frac{n}{2}\right) + n^2 \sin n$

violates regularity condition of case 3

Example 7: $T(n) = 2T\left(\frac{n}{2}\right) + \frac{n}{\log n}$

$f(n) = O(n^{\log_b a})$, but $\neq O(n^{\log_b a - \epsilon})$ for any constant $\epsilon > 0$

Example 8: $T(n) = T\left(\frac{n}{2}\right) + 2T\left(\frac{n}{4}\right) + n$

a and b are not fixed

Proof of The Master Theorem

Proof of the Master Theorem for Exact Powers of b

LEMMA 1: Let $a \geq 1$ and $b > 1$ be constants, and let $f(n)$ be a nonnegative function defined on exact powers of b . Define $T(n)$ on exact powers of b by the recurrence

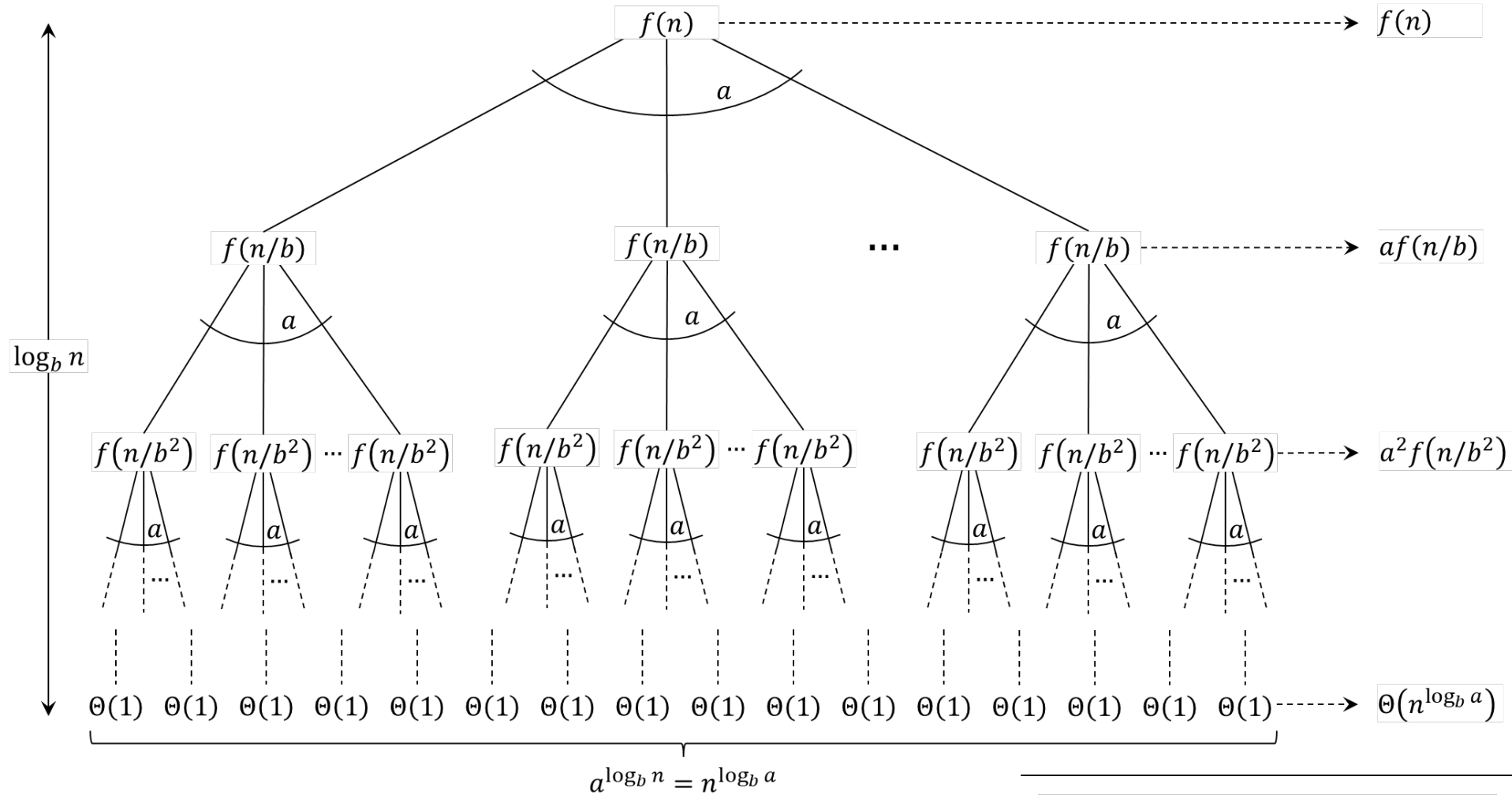
$$T(n) = \begin{cases} \Theta(1), & \text{if } n = 1, \\ aT\left(\frac{n}{b}\right) + f(n), & \text{if } n = b^i, \end{cases}$$

where i is a positive integer.

Then

$$T(n) = \Theta(n^{\log_b a}) + \sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right).$$

Proof of the Master Theorem for Exact Powers of b



$$\text{Total: } \Theta(n^{\log_b a}) + \sum_{j=0}^{\log_b n - 1} a^j f(n/b^j)$$

Proof of the Master Theorem for Exact Powers of b

LEMMA 2: Let $a \geq 1$ and $b > 1$ be constants, and let $f(n)$ be a nonnegative function defined on exact powers of b . A function $g(n)$ defined over exact powers of b by

$$g(n) = \sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right)$$

has the following asymptotic bounds for exact powers of b :

1. If $f(n) = O(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$, then
$$g(n) = O(n^{\log_b a}).$$
2. If $f(n) = \Theta(n^{\log_b a})$, then
$$g(n) = \Theta(n^{\log_b a} \log n).$$
3. If $af(n/b) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n , then
$$g(n) = \Theta(f(n)).$$

Proof of the Master Theorem for Exact Powers of b

PROOF OF LEMMA 2:

Case 1: We have:

$$f(n) = O(n^{\log_b a - \epsilon}) \Rightarrow f(n/b^j) = O\left((n/b^j)^{\log_b a - \epsilon}\right).$$

$$\text{Substituting: } g(n) = O\left(\sum_{j=0}^{\log_b n - 1} a^j \left(\frac{n}{b^j}\right)^{\log_b a - \epsilon}\right).$$

$$\begin{aligned} \text{Now, } \sum_{j=0}^{\log_b n - 1} a^j \left(\frac{n}{b^j}\right)^{\log_b a - \epsilon} &= n^{\log_b a - \epsilon} \sum_{j=0}^{\log_b n - 1} \left(\frac{ab^\epsilon}{b^{\log_b a}}\right)^j \\ &= n^{\log_b a - \epsilon} \sum_{j=0}^{\log_b n - 1} (b^\epsilon)^j \\ &= n^{\log_b a - \epsilon} \left(\frac{b^{\epsilon \log_b n} - 1}{b^\epsilon - 1}\right) \\ &= n^{\log_b a - \epsilon} \left(\frac{n^\epsilon - 1}{b^\epsilon - 1}\right) \\ &= n^{\log_b a - \epsilon} O(n^\epsilon) = O(n^{\log_b a}) \end{aligned}$$

Hence, $g(n) = O(n^{\log_b a})$.

Proof of the Master Theorem for Exact Powers of b

PROOF OF LEMMA 2:

Case 2: We have:

$$f(n) = \Theta(n^{\log_b a}) \Rightarrow f(n/b^j) = \Theta\left((n/b^j)^{\log_b a}\right).$$

$$\text{Substituting: } g(n) = \Theta\left(\sum_{j=0}^{\log_b n-1} a^j \left(\frac{n}{b^j}\right)^{\log_b a}\right).$$

$$\begin{aligned}\text{Now, } \sum_{j=0}^{\log_b n-1} a^j \left(\frac{n}{b^j}\right)^{\log_b a} &= n^{\log_b a} \sum_{j=0}^{\log_b n-1} \left(\frac{a}{b^{\log_b a}}\right)^j \\ &= n^{\log_b a} \sum_{j=0}^{\log_b n-1} 1 \\ &= n^{\log_b a} \log_b n\end{aligned}$$

$$\text{Hence, } g(n) = \Theta(n^{\log_b a} \log_b n) = \Theta(n^{\log_b a} \log n).$$

Proof of the Master Theorem for Exact Powers of b

PROOF OF LEMMA 2:

Case 3: Since $f(n)$ appears in the definition of $g(n)$ and all terms of $g(n)$ are nonnegative, we conclude that for exact powers of b :

$$g(n) = \Omega(f(n)).$$

Given that for some positive constant $c < 1$ and all $n \geq n_0$, where n_0 is another positive constant:

$$\begin{aligned}af\left(\frac{n}{b}\right) &\leq cf(n) \\ \Rightarrow f\left(\frac{n}{b}\right) &\leq \left(\frac{c}{a}\right)f(n) \\ \Rightarrow f\left(\frac{n}{b^j}\right) &\leq \left(\frac{c}{a}\right)^j f(n) \\ \Rightarrow a^j f\left(\frac{n}{b^j}\right) &\leq c^j f(n),\end{aligned}$$

where we assume that we iterate on values $\geq n_0$ only. Since the last, and smallest such value is $\frac{n}{b^{j-1}}$, it is enough to assume that $\frac{n}{b^{j-1}} \geq_{33} n_0$.

Proof of the Master Theorem for Exact Powers of b

PROOF OF LEMMA 2:

Case 3 (continued): From $\frac{n}{b^{j-1}} \geq n_0$ we have $j \leq \log_b \left(\frac{n}{n_0} \right) + 1$.

Hence, putting $j = \log_b \left(\frac{n}{n_0} \right) + 1$ in $a^j f \left(\frac{n}{b^j} \right) \leq c^j f(n)$, we get:

$$\begin{aligned} f(n) &\geq \left(\frac{a}{c} \right)^{\log_b \left(\frac{n}{n_0} \right) + 1} f \left(\frac{n}{b^{\log_b \left(\frac{n}{n_0} \right) + 1}} \right) \\ &= \frac{a}{c} \times \left(\frac{n}{n_0} \right)^{\log_b \left(\frac{a}{c} \right)} f \left(\frac{n_0}{b} \right) \\ &= \frac{a}{c} \times \frac{n^{\log_b a - \log_b c}}{n_0^{\log_b(a/c)}} \times \Theta(1) \\ &= n^{\log_b a + \epsilon} \times \Theta(1) = \Omega(n^{\log_b a + \epsilon}), \end{aligned}$$

where $\epsilon = -\log_b c > 0$ as $0 < c < 1$.

Proof of the Master Theorem for Exact Powers of b

PROOF OF LEMMA 2:

Case 3 (continued): Substituting into the expression for $g(n)$ we get:

$$\begin{aligned} g(n) &= \sum_{j=0}^{\log_b n - 1} a^j f(n/b^j) \\ &= \sum_{j=0}^{\log_b(n/n_0)+1} a^j f(n/b^j) + \sum_{j=\log_b(n/n_0)+2}^{\log_b n - 1} a^j f(n/b^j) \\ &\leq \sum_{j=0}^{\log_b(n/n_0)+1} c^j f(n) + \sum_{j=\log_b(n/n_0)+2}^{\log_b n - 1} a^j \times O(1) \\ &\leq f(n) \sum_{j=0}^{\infty} c^j + O(1) \sum_{j=0}^{\log_b n - 1} a^j = \frac{f(n)}{1-c} + \frac{a^{\log_b n} - 1}{a-1} O(1) \\ &= \Theta(f(n)) + O(n^{\log_b a}) = \Theta(f(n)) \Rightarrow g(n) = O(f(n)) \end{aligned}$$

Hence, for exact powers of b : $g(n) = \Theta(f(n))$.

Proof of the Master Theorem for Exact Powers of b

LEMMA 3: Let $a \geq 1$ and $b > 1$ be constants, and let $f(n)$ be a nonnegative function defined on exact powers of b . Define $T(n)$ on exact powers of b by the recurrence

$$T(n) = \begin{cases} \Theta(1), & \text{if } n = 1, \\ aT\left(\frac{n}{b}\right) + f(n), & \text{if } n = b^i, \end{cases}$$

where $i > 0$ is an integer.

Then $T(n)$ has the asymptotic bounds below for exact powers of b , and some constants $\epsilon > 0$ and $c < 0$:

1. If $f(n) = O(n^{\log_b a - \epsilon})$, then $T(n) = \Theta(n^{\log_b a})$.
2. If $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \log n)$.
3. If $f(n) = \Omega(n^{\log_b a + \epsilon})$, and if $af(n/b) \leq cf(n)$ for all sufficiently large n , then $T(n) = \Theta(f(n))$.

PROOF OF LEMMA 3: Follows from Lemma 1 and Lemma 2.

Optional

**Extending the Master Theorem
to All Integral n**

Extending the Master Theorem to All Integral n

We need to extend our analysis to allow situations in which floors and ceilings appear in the Master recurrence:

$$T(n) = aT\left(\left\lceil \frac{n}{b} \right\rceil\right) + f(n) \quad \dots\dots (1)$$

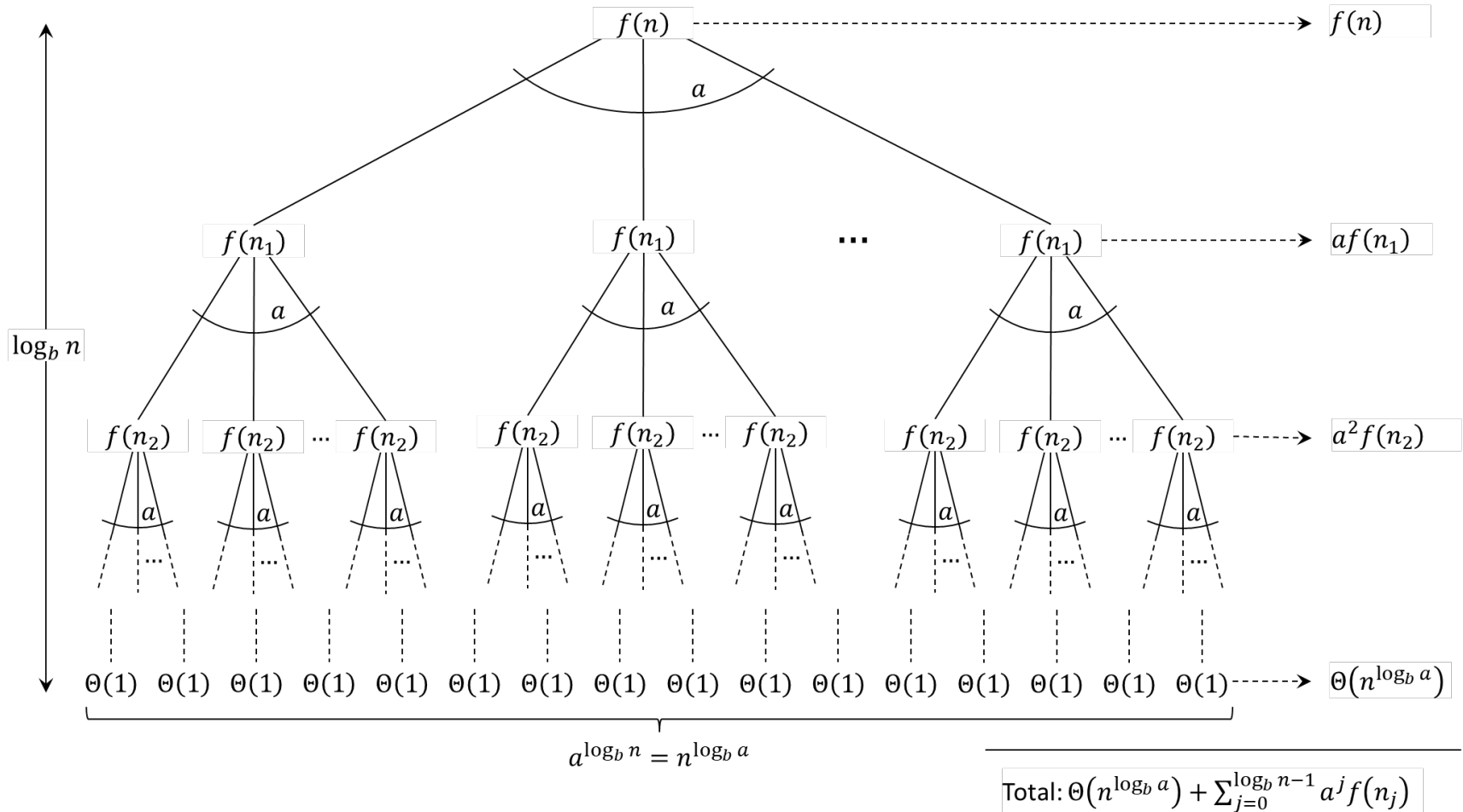
$$\text{and } T(n) = aT\left(\left\lfloor \frac{n}{b} \right\rfloor\right) + f(n) \quad \dots\dots (2)$$

Obtaining a lower bound on recurrence (1) and an upper bound on recurrence (2) are not difficult because we can use $\left\lceil \frac{n}{b} \right\rceil \geq \frac{n}{b}$ in the first case and $\left\lfloor \frac{n}{b} \right\rfloor \leq \frac{n}{b}$ in the second case.

Upper bounding recurrence (1) and lower bounding recurrence (2) require more effort, but they use similar techniques.

So, we will only try to prove an upper bound on recurrence (1).

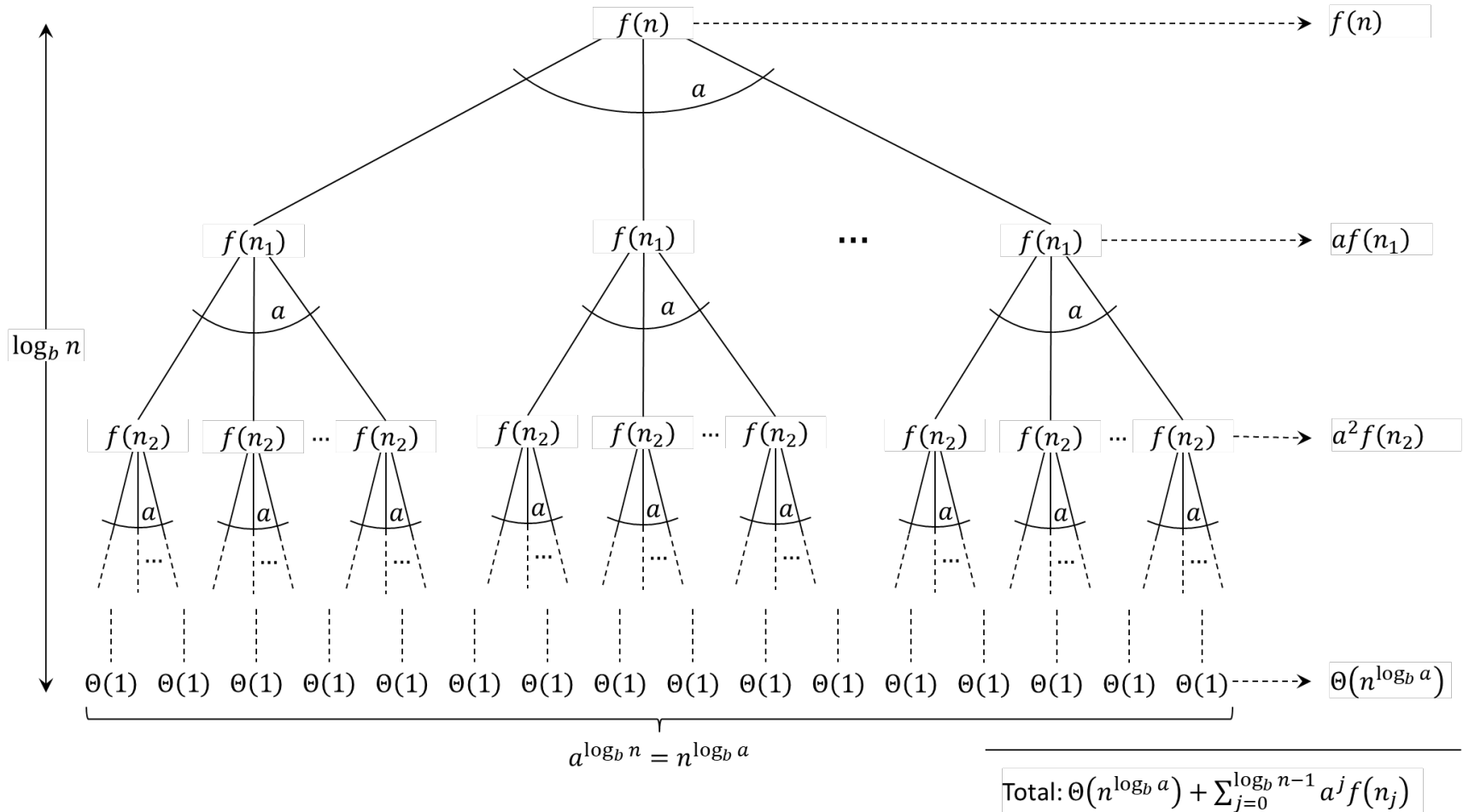
Upper Bounding $T(n) = aT[n/b] + f(n)$



As we go down the recursion tree we encounter a sequence of recursive invocations on the arguments:

$$n, \quad \lceil n/b \rceil, \quad \lceil \lceil n/b \rceil / b \rceil, \quad \lceil \lceil \lceil n/b \rceil / b \rceil / b \rceil, \quad \dots \dots \dots$$

Upper Bounding $T(n) = aT[n/b] + f(n)$



Let's define the j^{th} element in the sequence by n_j , where

$$n_j = \begin{cases} n, & \text{if } j = 0, \\ \lceil n_{j-1}/b \rceil, & \text{if } j > 0. \end{cases}$$

Upper Bounding $T(n) = aT[n/b] + f(n)$

Let's first determine a depth h such that n_h is a constant.

Using the inequality $[x] \leq x + 1$, we obtain:

$$n_0 \leq n,$$

$$n_1 \leq \frac{n}{b} + 1,$$

$$n_2 \leq \frac{n}{b^2} + \frac{1}{b} + 1,$$

$$n_3 \leq \frac{n}{b^3} + \frac{1}{b^2} + \frac{1}{b} + 1,$$

and so on.

In general,

$$\begin{aligned} n_j &\leq \frac{n}{b^j} + \sum_{i=0}^{j-1} \frac{1}{b^i} \\ &< \frac{n}{b^j} + \sum_{i=0}^{\infty} \frac{1}{b^i} \\ &= \frac{n}{b^j} + \frac{b}{b-1}. \end{aligned}$$

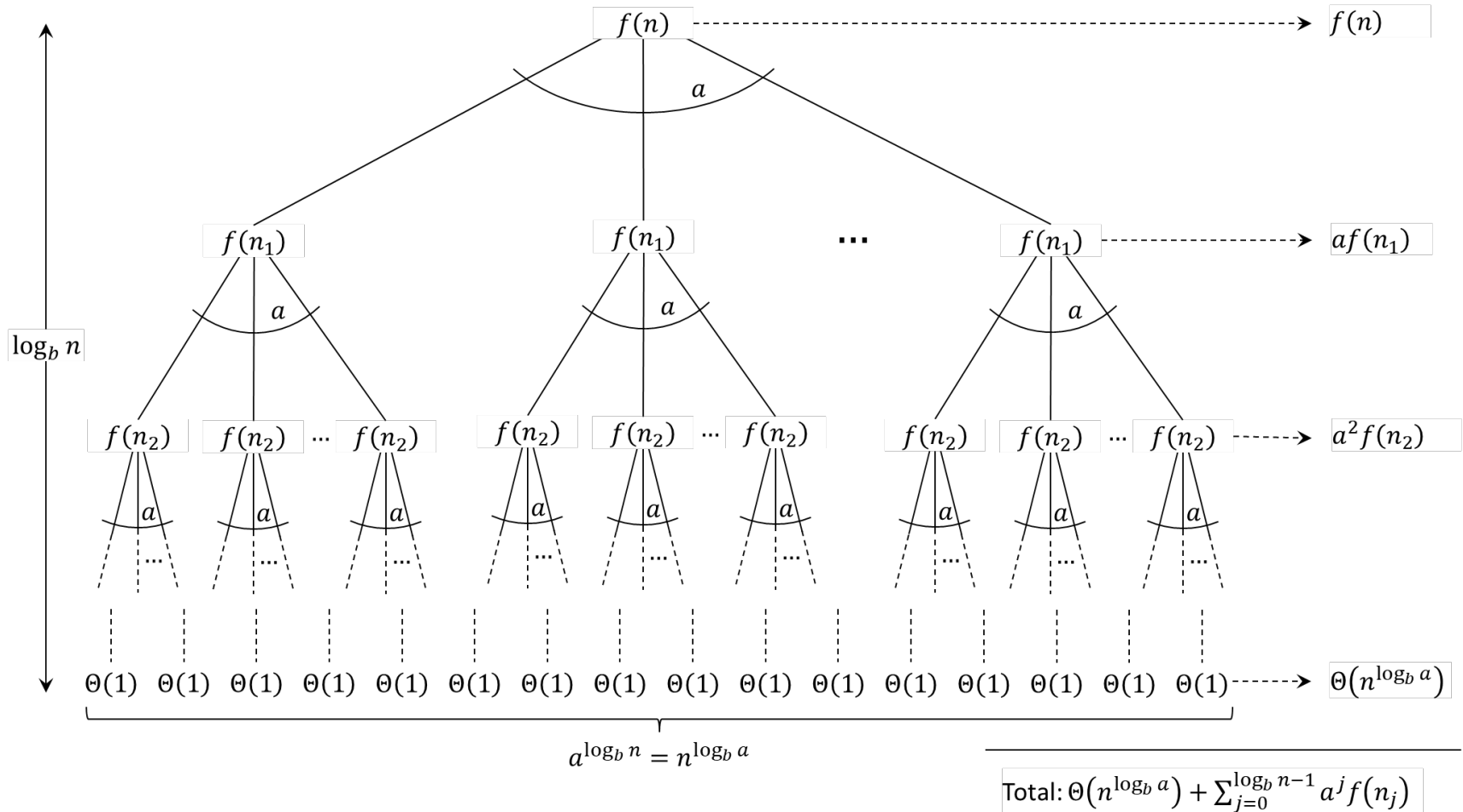
Upper Bounding $T(n) = aT[n/b] + f(n)$

Letting $h = \lfloor \log_b n \rfloor$ we obtain:

$$\begin{aligned} n_{\lfloor \log_b n \rfloor} &< \frac{n}{b^{\lfloor \log_b n \rfloor}} + \frac{b}{b-1} \\ &< \frac{n}{b^{\log_b n - 1}} + \frac{b}{b-1} \\ &< \frac{n}{n/b} + \frac{b}{b-1} \\ &= b + \frac{b}{b-1} \\ &= O(1) \end{aligned}$$

Hence, at depth $h = \lfloor \log_b n \rfloor$ the problem size is at most a constant.

Upper Bounding $T(n) = aT[n/b] + f(n)$



From the figure above we get:

$$T(n) = \Theta(n^{\log_b a}) + \sum_{j=0}^{\log_b n - 1} a^j f(n_j)$$

Upper Bounding $T(n) = aT[n/b] + f(n)$

We have:

$$T(n) = \Theta(n^{\log_b a}) + \sum_{j=0}^{\log_b n - 1} a^j f(n_j)$$

We will have to evaluate the following sum:

$$g(n) = \sum_{j=0}^{\log_b n - 1} a^j f(n_j)$$

Upper Bounding $T(n) = aT[n/b] + f(n)$

We will evaluate the following sum:

$$g(n) = \sum_{j=0}^{\log_b n - 1} a^j f(n_j)$$

Case 2: We have: $f(n) = \Theta(n^{\log_b a})$.

If we can show that $f(n_j) = O\left(\left(\frac{n}{b^j}\right)^{\log_b a}\right)$, then case 2 of Lemma 2 will go through.

Observe that $j \leq \lfloor \log_b n \rfloor \Rightarrow \frac{b^j}{n} \leq 1$.

Also, $f(n) = O(n^{\log_b a})$ implies that there exists a constant $c' > 0$ such that for all sufficiently large n_j the following holds:

$$f(n_j) \leq c' \left(\frac{n}{b^j} + \frac{b}{b-1}\right)^{\log_b a}.$$

Upper Bounding $T(n) = aT[n/b] + f(n)$

Case 2 (continued): We have:

$$\begin{aligned} f(n_j) &\leq c' \left(\frac{n}{b^j} + \frac{b}{b-1} \right)^{\log_b a} \\ &= c' \left(\frac{n}{b^j} \left(1 + \frac{b^j}{n} \frac{b}{b-1} \right) \right)^{\log_b a} . \\ &= c' \left(\frac{n^{\log_b a}}{a^j} \right) \left(1 + \frac{b^j}{n} \frac{b}{b-1} \right)^{\log_b a} . \\ &\leq c' \left(\frac{n^{\log_b a}}{a^j} \right) \left(1 + \frac{b}{b-1} \right)^{\log_b a} . \\ &= O \left(\frac{n^{\log_b a}}{a^j} \right) \\ &= O \left(\left(\frac{n}{b^j} \right)^{\log_b a} \right) \end{aligned}$$

Upper Bounding $T(n) = aT[n/b] + f(n)$

Case 1: The proof is similar to that of case 2. The key is to prove the bound $f(n_j) = O\left(\left(\frac{n}{b^j}\right)^{\log_b a - \epsilon}\right)$ which is similar to what we did in case 2 though the algebra is more intricate.

Upper Bounding $T(n) = aT[n/b] + f(n)$

Case 3: If $af\left(\left\lceil\frac{n}{b}\right\rceil\right) \leq cf(n)$ for $n > b + \frac{b}{b-1}$, where $c < 1$ is a constant then it follows that $a^j f(n_j) \leq c^j f(n)$.

Therefore, we can evaluate $g(n)$ as in the proof of Lemma 2.

Fractals and Recursive Divide and Conquer

Here is the recurrence relation describing the running time of **Karatsuba's algorithm** for multiplying two n -bit binary integers:

$$T(n) = \begin{cases} \Theta(n^2), & \text{if } n \leq m, \\ 3T\left(\frac{n}{2}\right) + \Theta(n), & \text{otherwise.} \end{cases}$$

Since this falls under **Case 1** of the **Master Theorem**, base case computations dominate the running time. Therefore, we focus only on the recursive part of the recurrence:

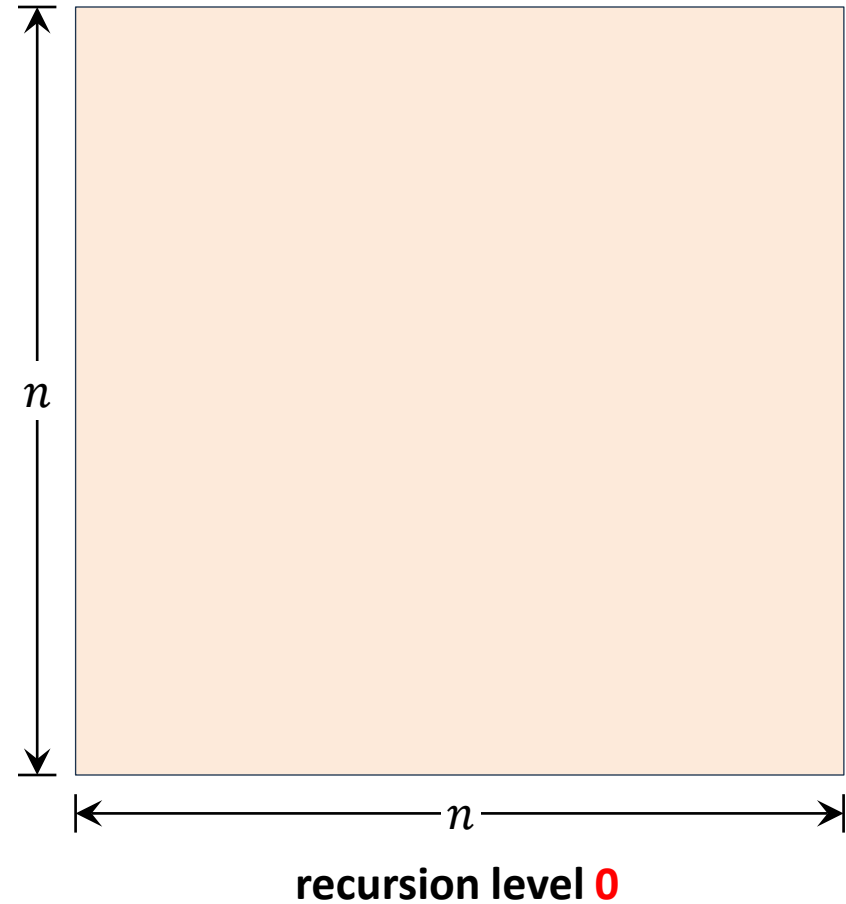
$$T(n) = \begin{cases} n^2, & \text{if } n \leq m, \\ 3T\left(\frac{n}{2}\right), & \text{otherwise.} \end{cases}$$

Fractals and Recursive Divide and Conquer

$$T(n) = \begin{cases} n^2, & \text{if } n \leq m, \\ 3T\left(\frac{n}{2}\right), & \text{otherwise.} \end{cases}$$

Using base case at recursion level 0:

$$T(n) = n^2$$

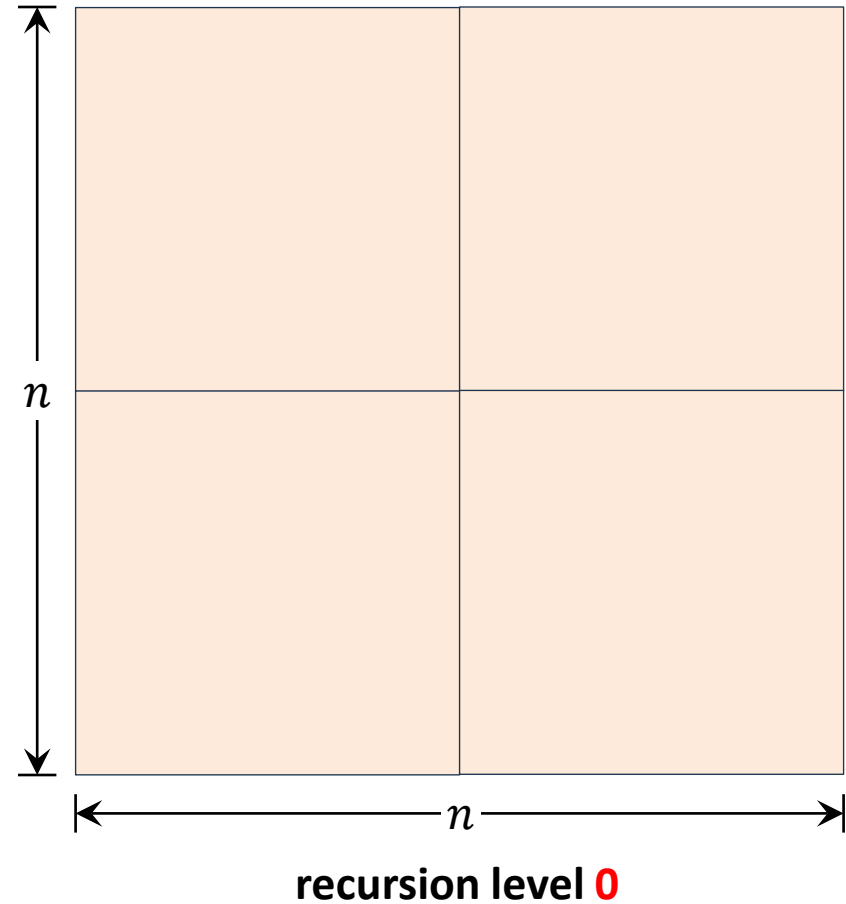


Fractals and Recursive Divide and Conquer

$$T(n) = \begin{cases} n^2, & \text{if } n \leq m, \\ 3T\left(\frac{n}{2}\right), & \text{otherwise.} \end{cases}$$

Using base case at recursion level 0:

$$T(n) = n^2$$

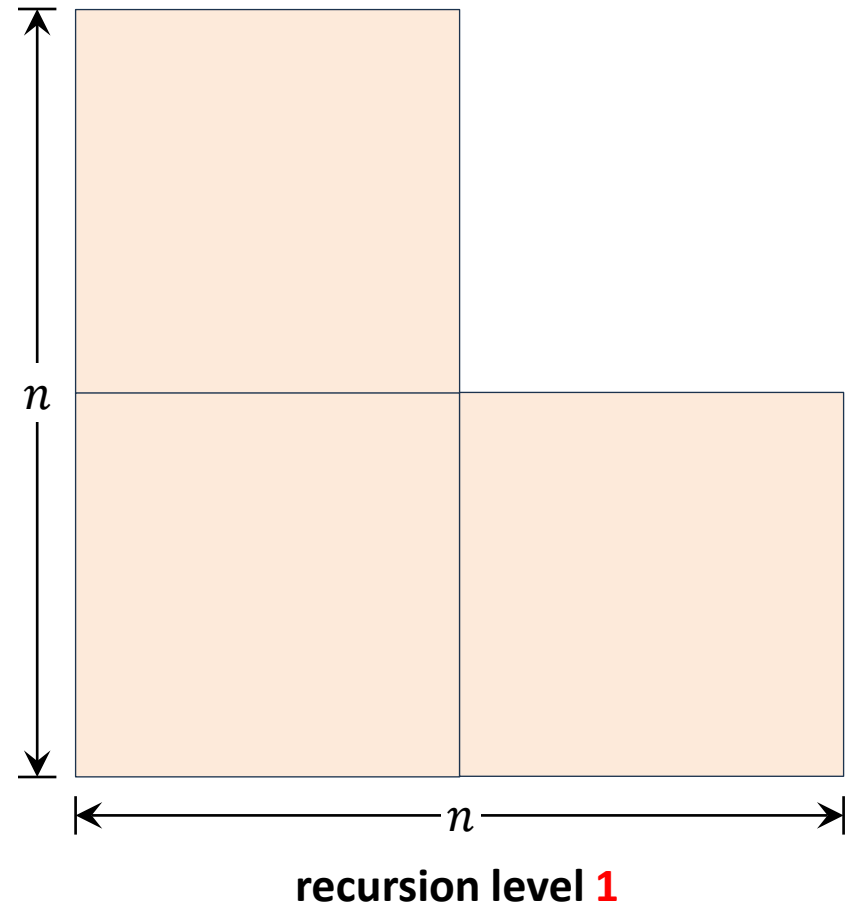


Fractals and Recursive Divide and Conquer

$$T(n) = \begin{cases} n^2, & \text{if } n \leq m, \\ 3T\left(\frac{n}{2}\right), & \text{otherwise.} \end{cases}$$

Using base case at recursion level 1:

$$T(n) = \frac{3}{4}n^2$$

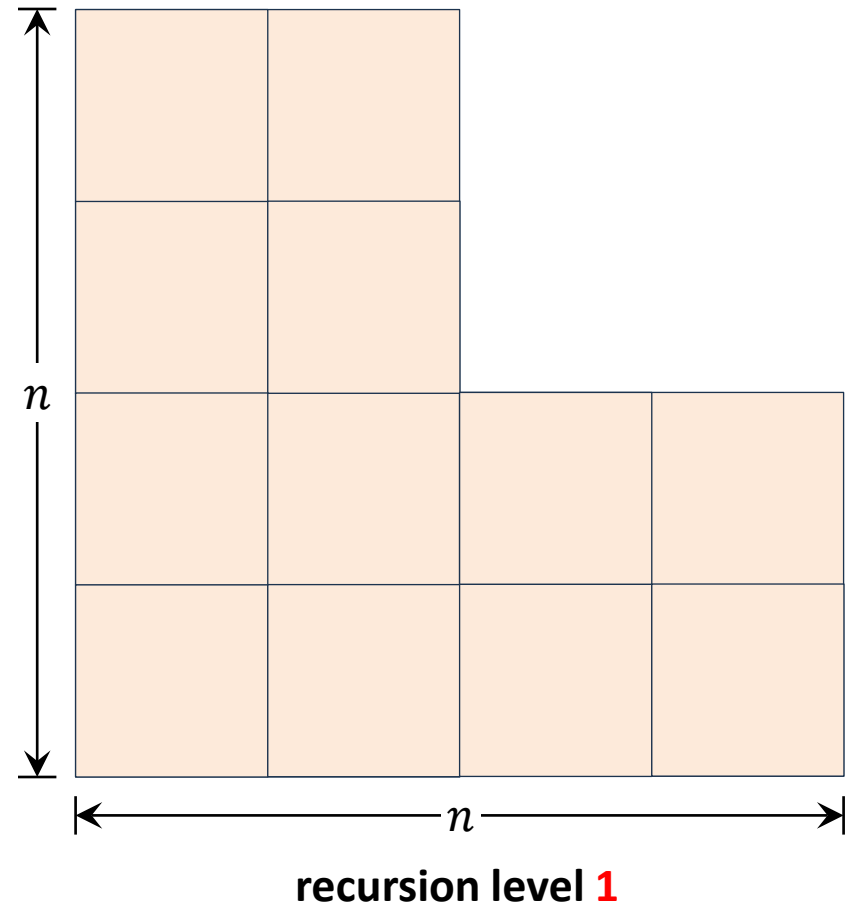


Fractals and Recursive Divide and Conquer

$$T(n) = \begin{cases} n^2, & \text{if } n \leq m, \\ 3T\left(\frac{n}{2}\right), & \text{otherwise.} \end{cases}$$

Using base case at recursion level 1:

$$T(n) = \frac{3}{4}n^2$$

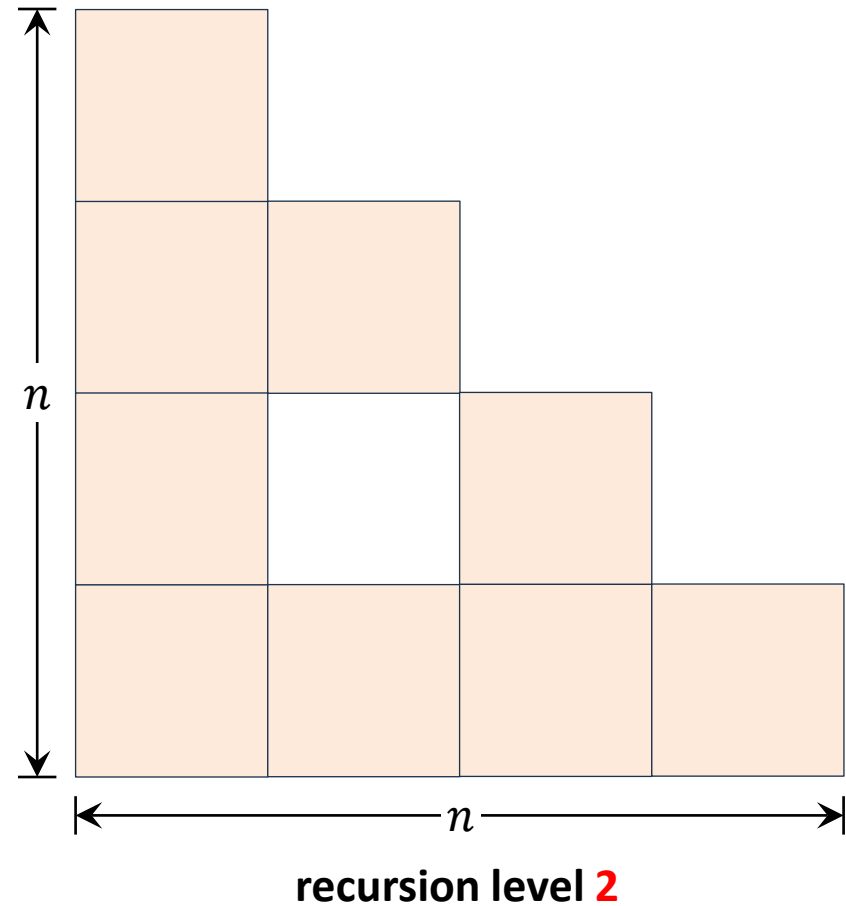


Fractals and Recursive Divide and Conquer

$$T(n) = \begin{cases} n^2, & \text{if } n \leq m, \\ 3T\left(\frac{n}{2}\right), & \text{otherwise.} \end{cases}$$

Using base case at recursion level 2:

$$T(n) = \frac{3^2}{4^2} n^2 = \frac{9}{16} n^2$$

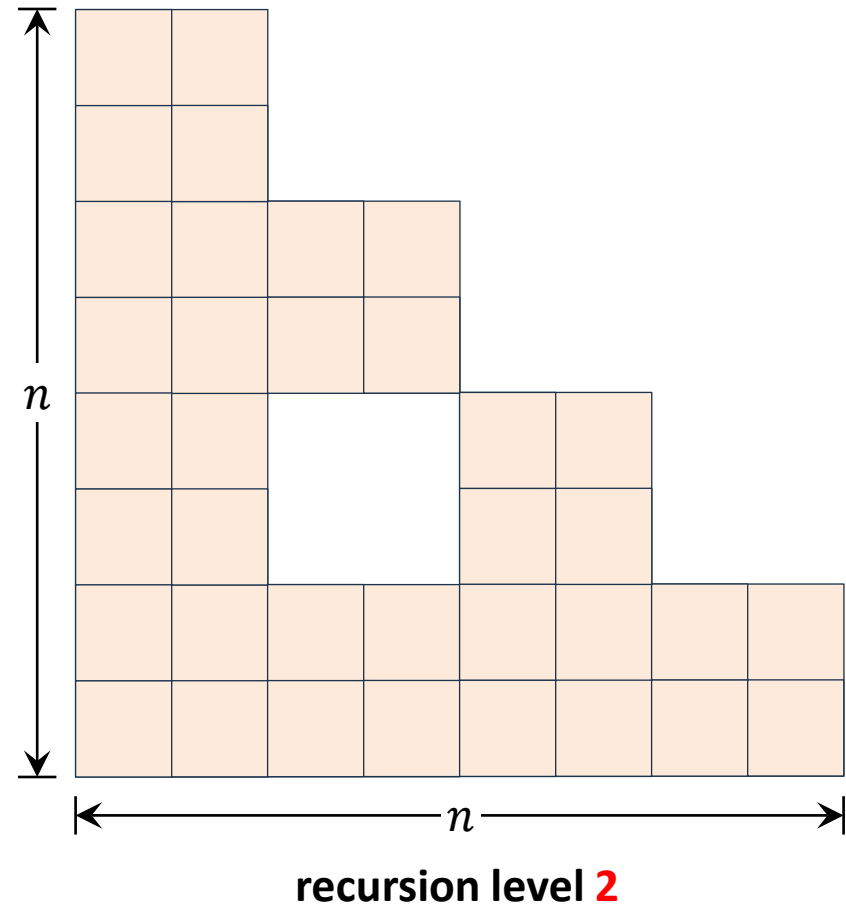


Fractals and Recursive Divide and Conquer

$$T(n) = \begin{cases} n^2, & \text{if } n \leq m, \\ 3T\left(\frac{n}{2}\right), & \text{otherwise.} \end{cases}$$

Using base case at recursion level 2:

$$T(n) = \frac{3^2}{4^2} n^2 = \frac{9}{16} n^2$$

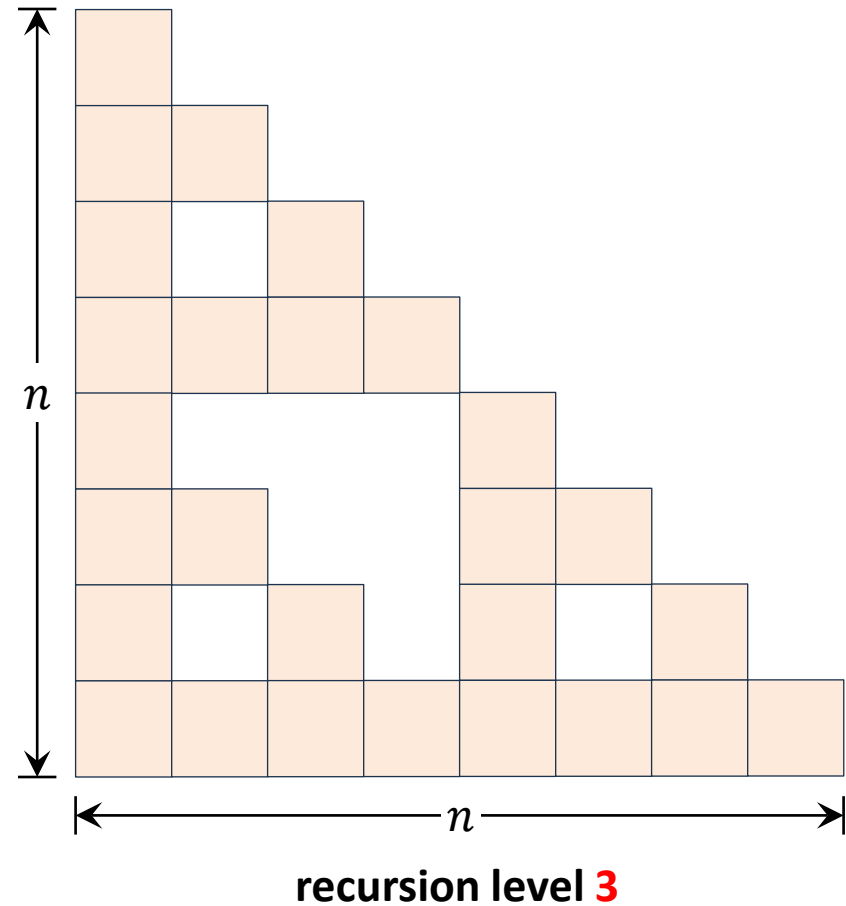


Fractals and Recursive Divide and Conquer

$$T(n) = \begin{cases} n^2, & \text{if } n \leq m, \\ 3T\left(\frac{n}{2}\right), & \text{otherwise.} \end{cases}$$

Using base case at recursion level 3:

$$T(n) = \frac{3^3}{4^3} n^2 = \frac{27}{64} n^2 < \frac{1}{2} n^2$$

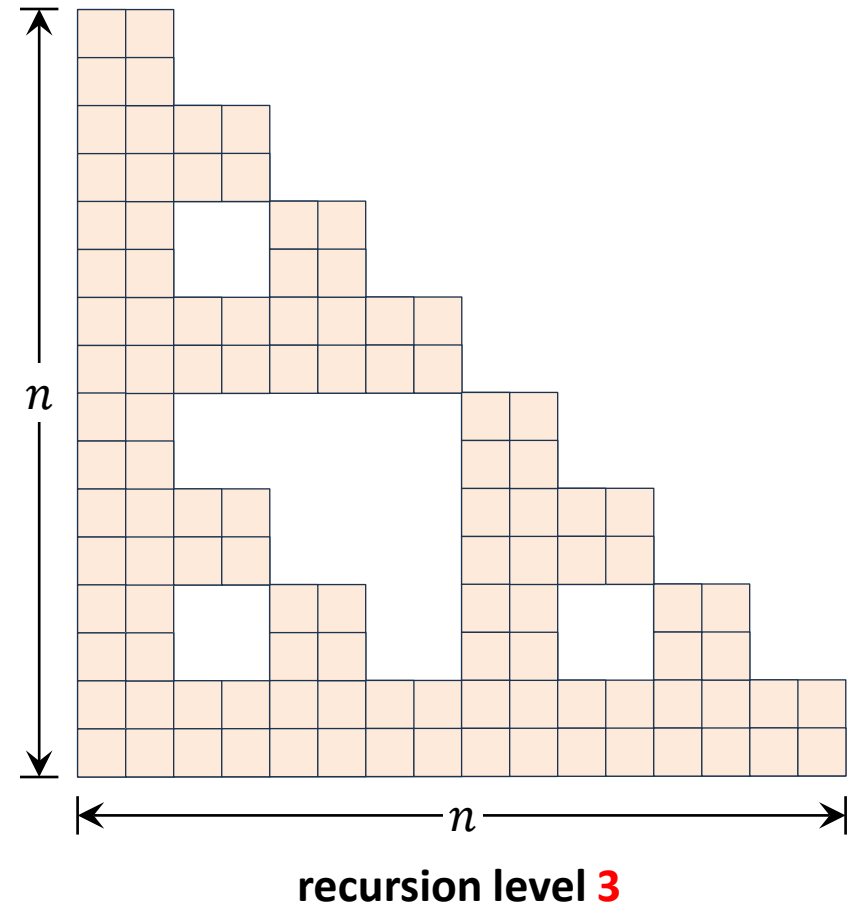


Fractals and Recursive Divide and Conquer

$$T(n) = \begin{cases} n^2, & \text{if } n \leq m, \\ 3T\left(\frac{n}{2}\right), & \text{otherwise.} \end{cases}$$

Using base case at recursion level 3:

$$T(n) = \frac{3^3}{4^3} n^2 = \frac{27}{64} n^2 < \frac{1}{2} n^2$$

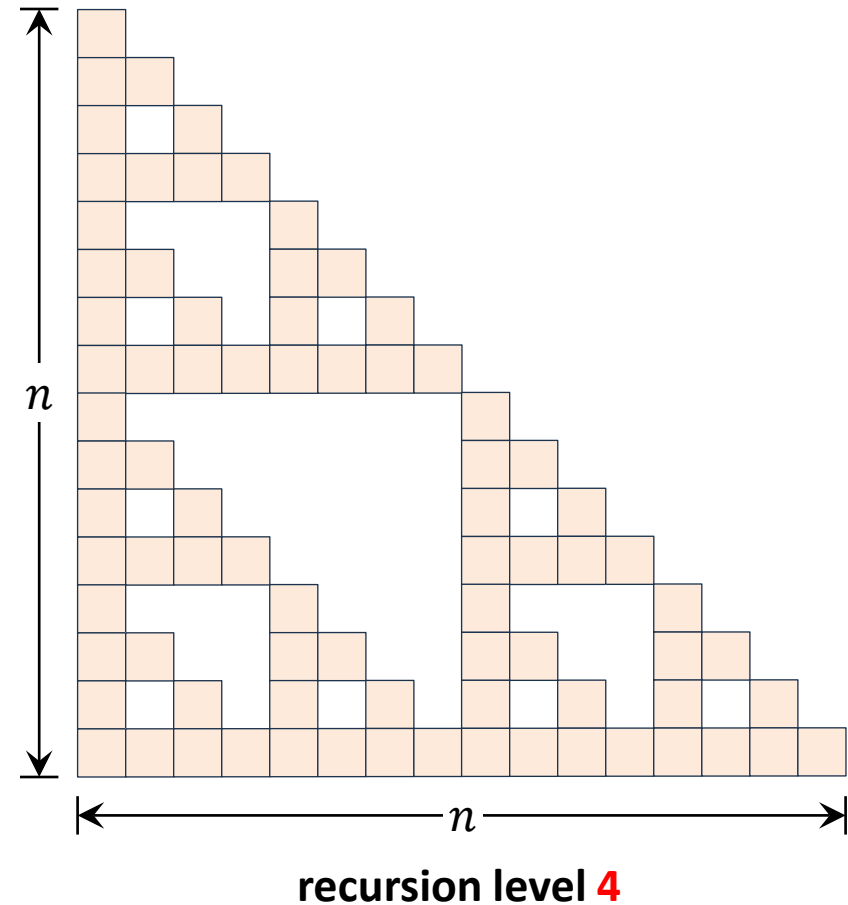


Fractals and Recursive Divide and Conquer

$$T(n) = \begin{cases} n^2, & \text{if } n \leq m, \\ 3T\left(\frac{n}{2}\right), & \text{otherwise.} \end{cases}$$

Using base case at recursion level 4:

$$T(n) = \frac{3^4}{4^4} n^2 = \frac{81}{256} n^2 < \frac{1}{3} n^2$$

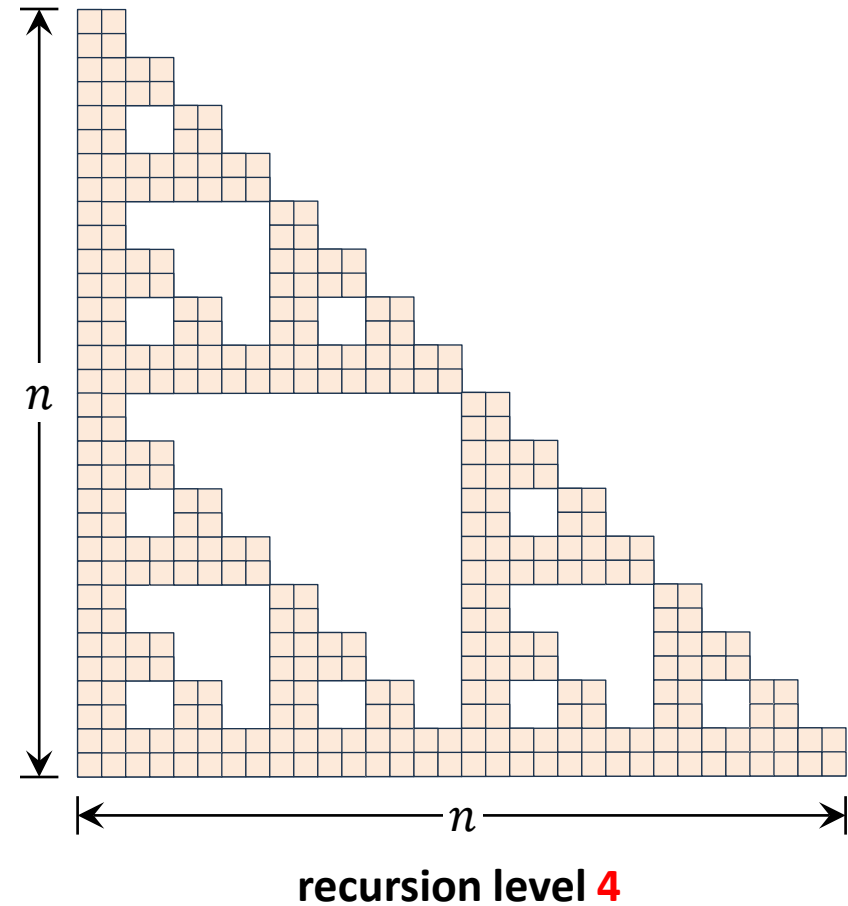


Fractals and Recursive Divide and Conquer

$$T(n) = \begin{cases} n^2, & \text{if } n \leq m, \\ 3T\left(\frac{n}{2}\right), & \text{otherwise.} \end{cases}$$

Using base case at recursion level 4:

$$T(n) = \frac{3^4}{4^4} n^2 = \frac{81}{256} n^2 < \frac{1}{3} n^2$$

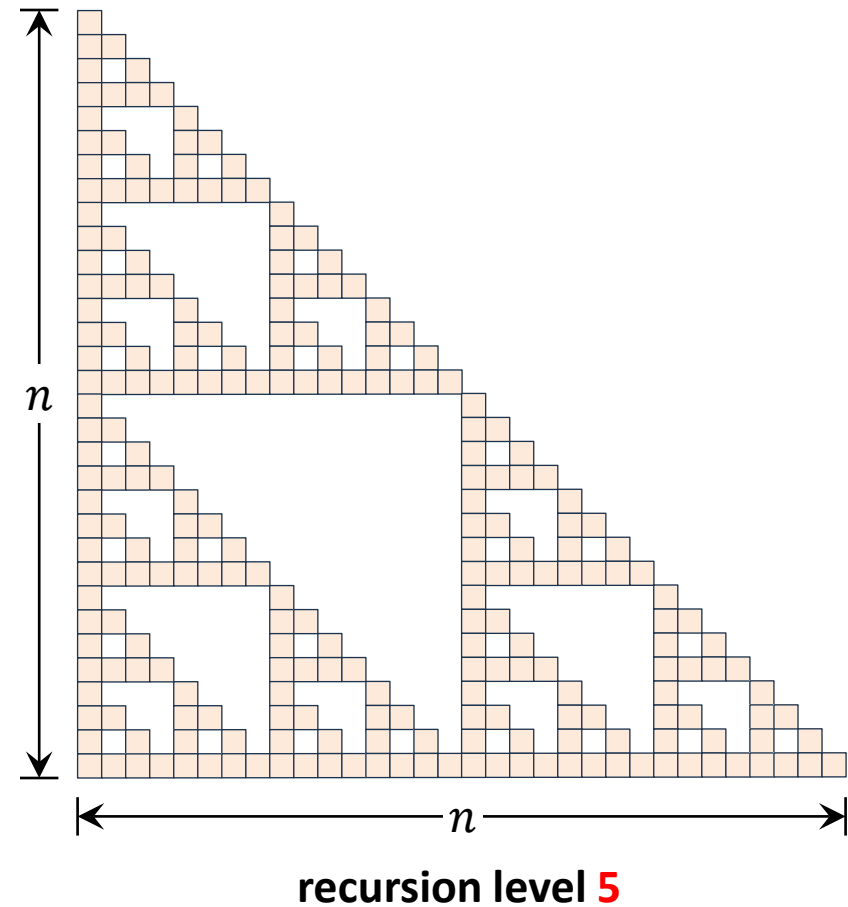


Fractals and Recursive Divide and Conquer

$$T(n) = \begin{cases} n^2, & \text{if } n \leq m, \\ 3T\left(\frac{n}{2}\right), & \text{otherwise.} \end{cases}$$

Using base case at recursion level 5:

$$T(n) = \frac{3^5}{4^5} n^2 = \frac{243}{1024} n^2 < \frac{1}{4} n^2$$

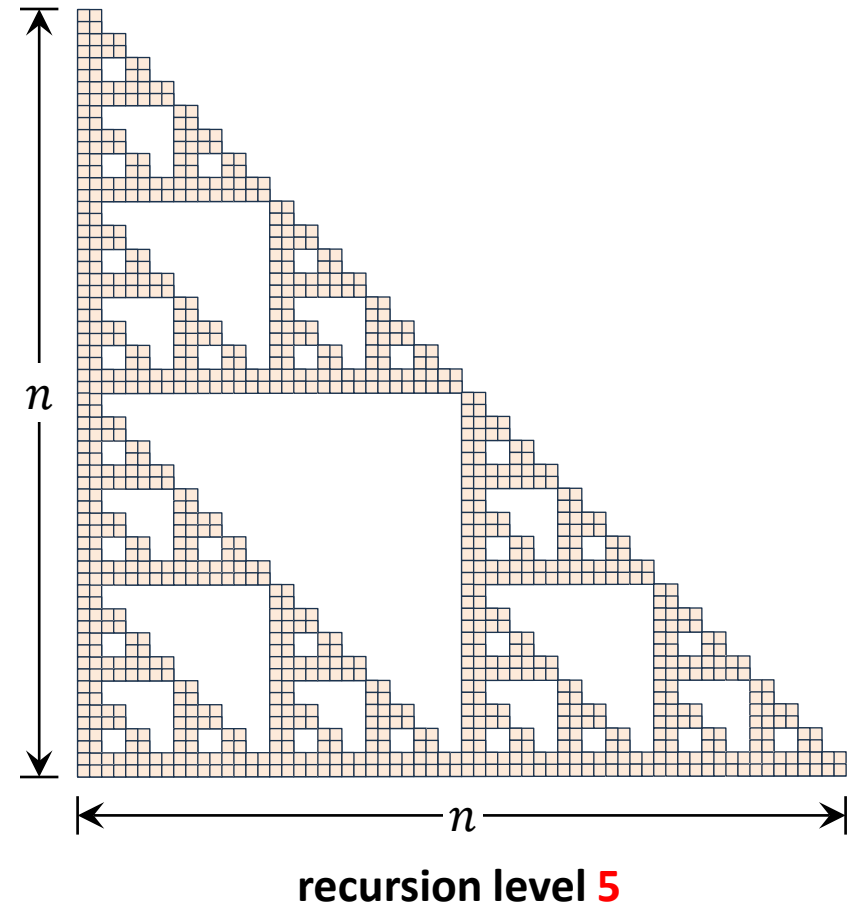


Fractals and Recursive Divide and Conquer

$$T(n) = \begin{cases} n^2, & \text{if } n \leq m, \\ 3T\left(\frac{n}{2}\right), & \text{otherwise.} \end{cases}$$

Using base case at recursion level 5:

$$T(n) = \frac{3^5}{4^5} n^2 = \frac{243}{1024} n^2 < \frac{1}{4} n^2$$

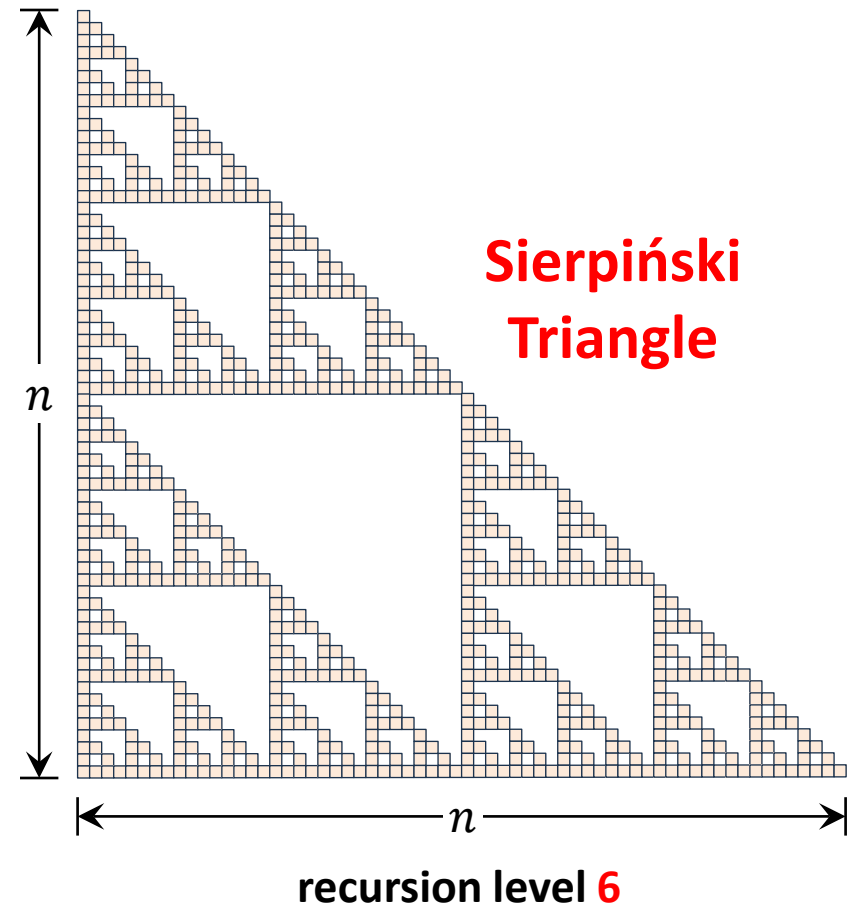


Fractals and Recursive Divide and Conquer

$$T(n) = \begin{cases} n^2, & \text{if } n \leq m, \\ 3T\left(\frac{n}{2}\right), & \text{otherwise.} \end{cases}$$

Using base case at recursion level 6:

$$T(n) = \frac{3^6}{4^6} n^2 = \frac{729}{4096} n^2 < \frac{1}{5} n^2$$



Fractals and Recursive Divide and Conquer

Here is the recurrence relation describing the running time of **Strassen's algorithm** for multiplying two $n \times n$ matrices:

$$T(n) = \begin{cases} \Theta(n^3), & \text{if } n \leq m, \\ 7T\left(\frac{n}{2}\right) + \Theta(n^2), & \text{otherwise.} \end{cases}$$

Since this falls under **Case 1** of the **Master Theorem**, base case computations dominate the running time. Therefore, we focus only on the recursive part of the recurrence:

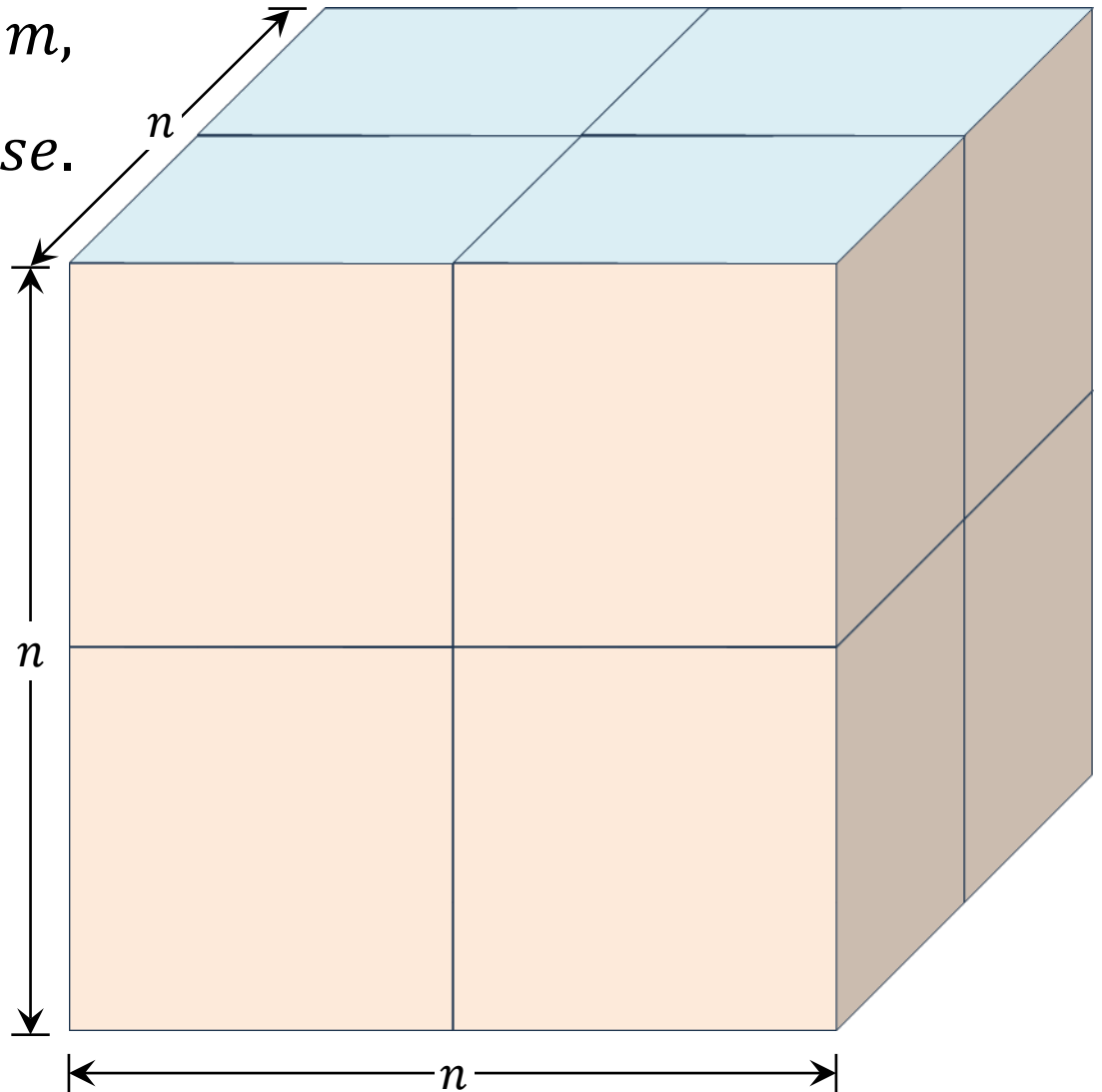
$$T(n) = \begin{cases} n^3, & \text{if } n \leq m, \\ 7T\left(\frac{n}{2}\right), & \text{otherwise.} \end{cases}$$

Fractals and Recursive Divide and Conquer

$$T(n) = \begin{cases} n^3, & \text{if } n \leq m, \\ 7T\left(\frac{n}{2}\right), & \text{otherwise.} \end{cases}$$

Using base case at
recursion level 0:

$$T(n) = n^3$$



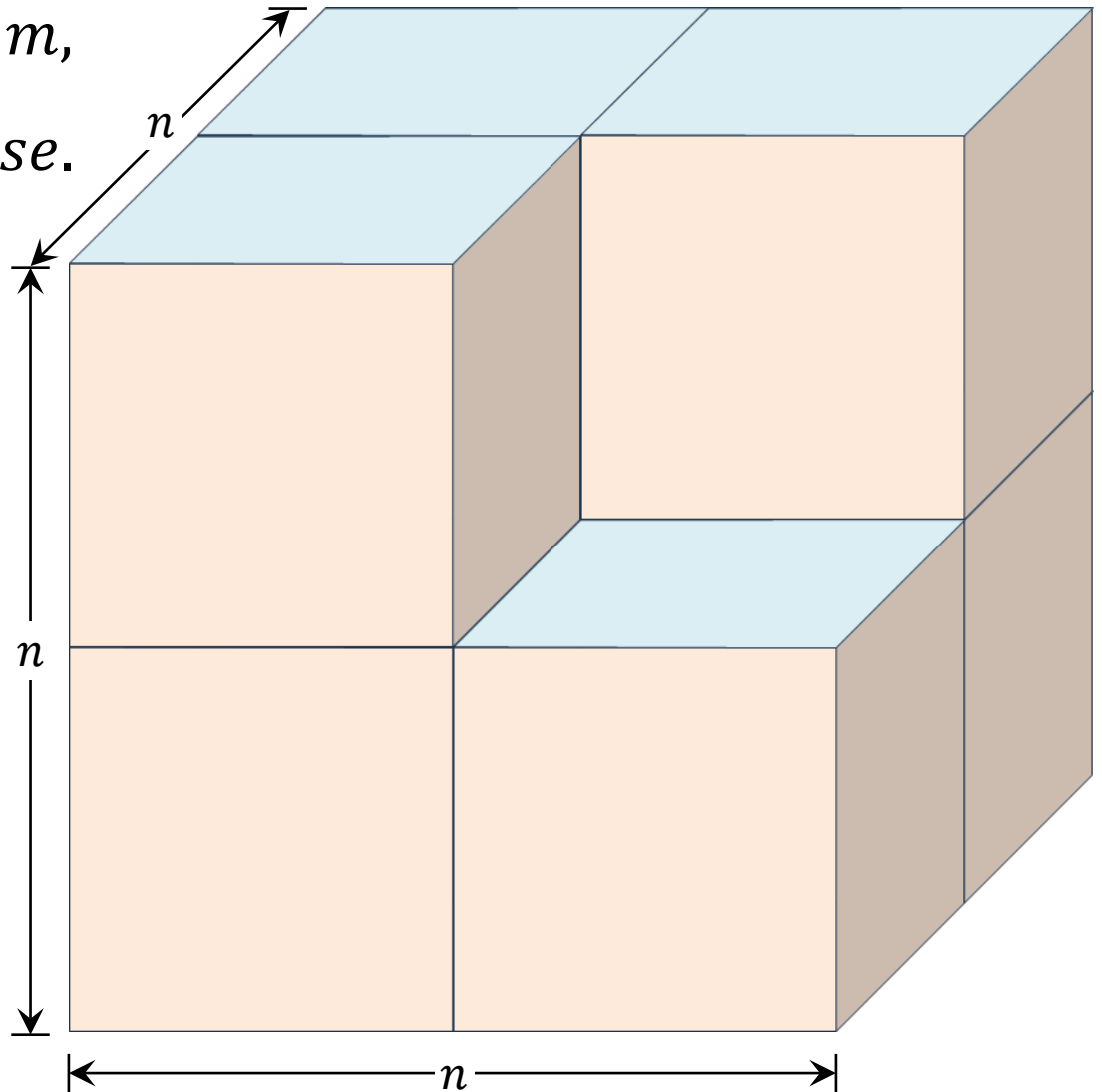
recursion level 0

Fractals and Recursive Divide and Conquer

$$T(n) = \begin{cases} n^3, & \text{if } n \leq m, \\ 7T\left(\frac{n}{2}\right), & \text{otherwise.} \end{cases}$$

Using base case at
recursion level 1:

$$T(n) = \frac{7}{8}n^3$$



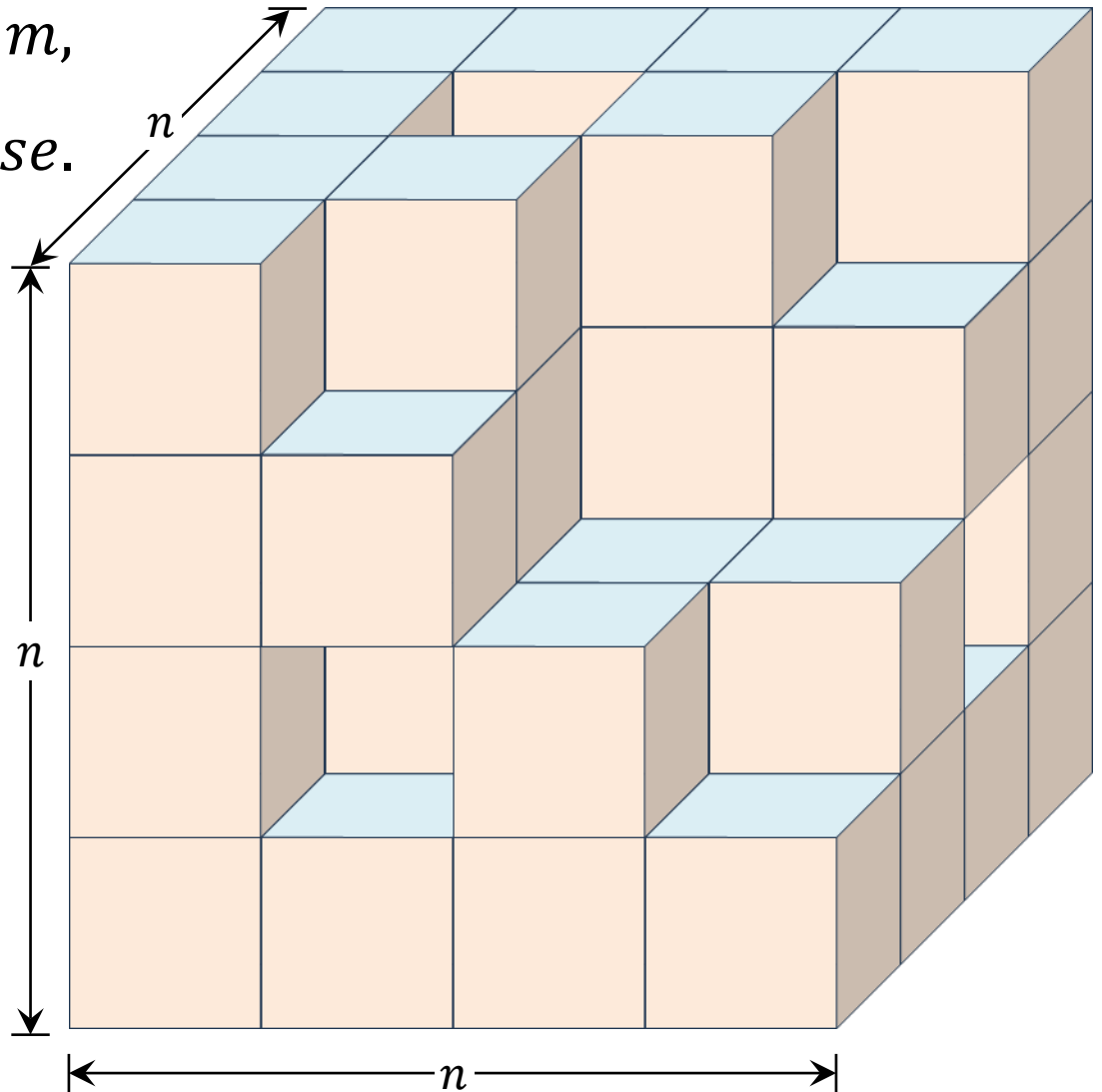
recursion level **1**

Fractals and Recursive Divide and Conquer

$$T(n) = \begin{cases} n^3, & \text{if } n \leq m, \\ 7T\left(\frac{n}{2}\right), & \text{otherwise.} \end{cases}$$

Using base case at
recursion level 2:

$$T(n) = \frac{7^2}{8^2} n^3$$



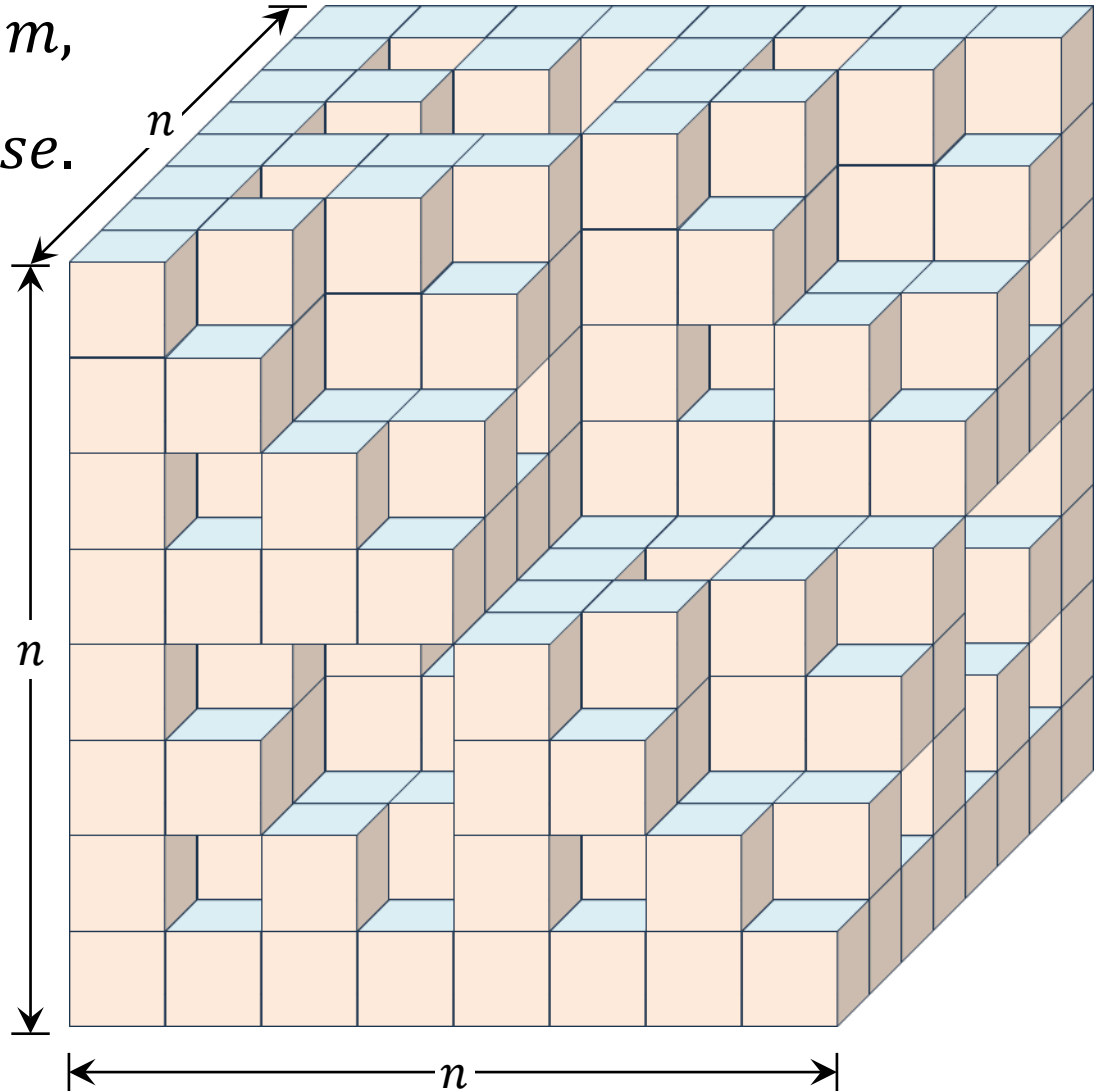
recursion level 2

Fractals and Recursive Divide and Conquer

$$T(n) = \begin{cases} n^3, & \text{if } n \leq m, \\ 7T\left(\frac{n}{2}\right), & \text{otherwise.} \end{cases}$$

Using base case at
recursion level 3:

$$T(n) = \frac{7^3}{8^3} n^3$$



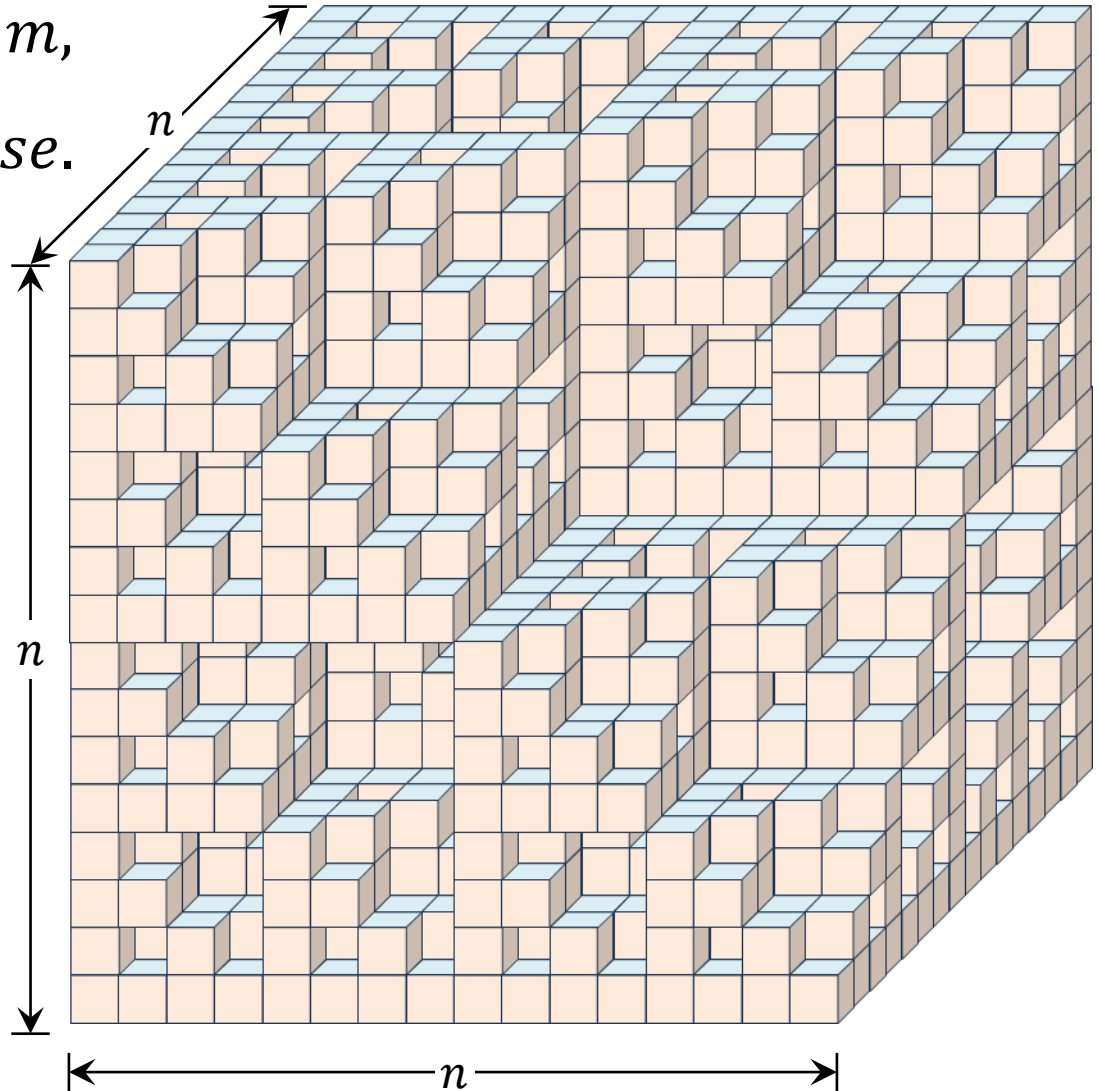
recursion level 3

Fractals and Recursive Divide and Conquer

$$T(n) = \begin{cases} n^3, & \text{if } n \leq m, \\ 7T\left(\frac{n}{2}\right), & \text{otherwise.} \end{cases}$$

Using base case at
recursion level 4:

$$T(n) = \frac{7^4}{8^4} n^3$$



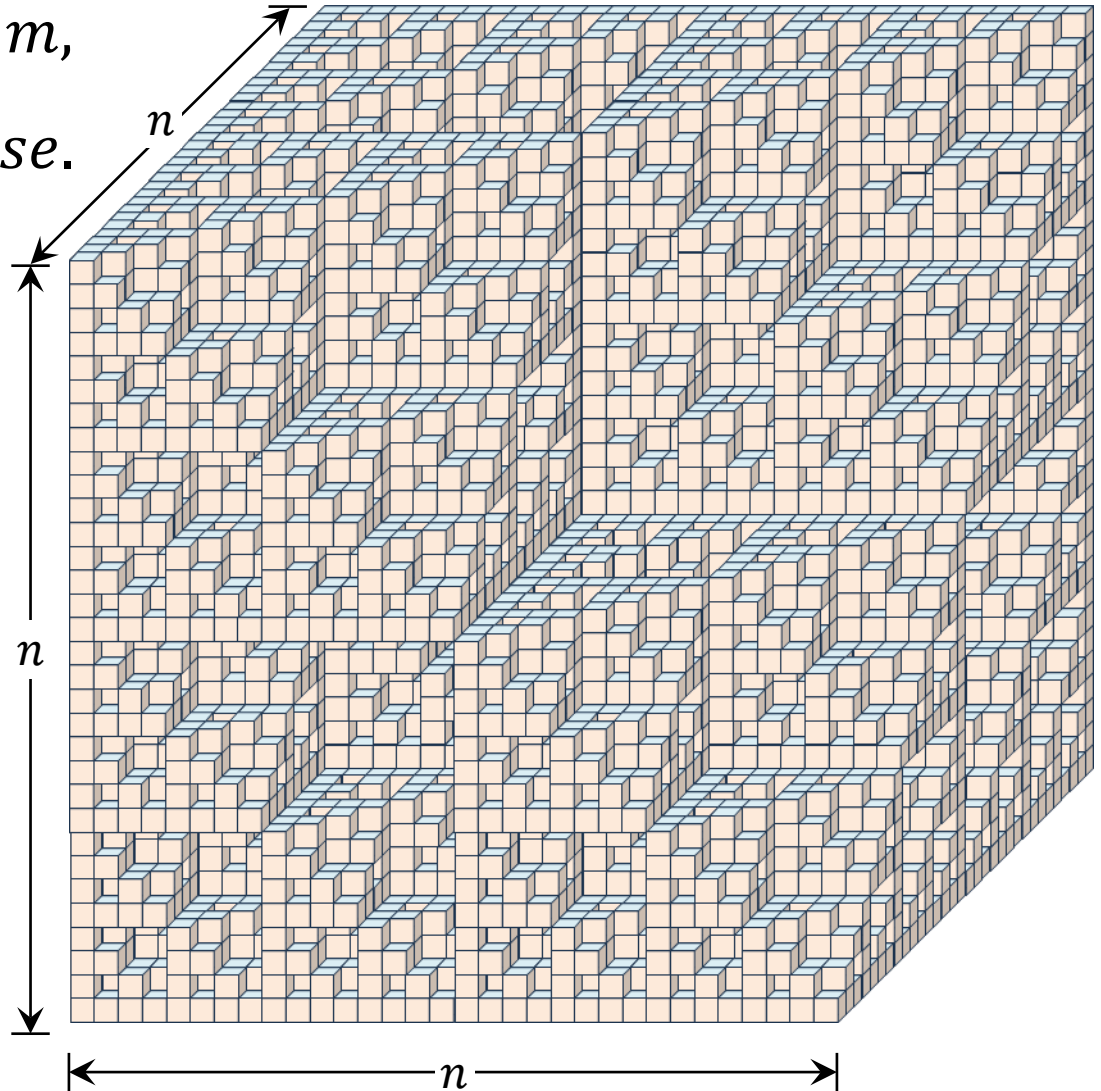
recursion level 4

Fractals and Recursive Divide and Conquer

$$T(n) = \begin{cases} n^3, & \text{if } n \leq m, \\ 7T\left(\frac{n}{2}\right), & \text{otherwise.} \end{cases}$$

Using base case at
recursion level 5:

$$T(n) = \frac{7^5}{8^5} n^3$$
$$\approx \frac{1}{2} n^3$$



recursion level **5**